

# Claude Certified Architect — תעודת Foundations

## מדריך לימוד (מבוסס על מדריך המבחן הרשמי)

### הקדמה

(trade-off) מאשרת שמומחה מסוגל לקבל החלטות מאוזנות Claude Certified Architect — Foundations תעודת Claude. בעת מימוש פתרונות אמיתיים מבוססי (trade-off) Claude Code-המבחן בוחן ידע יסודי ב Claude Agent SDK-ב, Claude Code-המבחן בוחן ידע יסודי ב Claude. הטכנולוגיות המרכזיות לבניית אפליקציות פרודקשן עם — Model Context Protocol (MCP) וב Claude API-ב Claude.

לתמיכת לקוחות, תכנון (agentic) שאלות המבחן מבוססות על תרחישי תעשייה ריאליסטיים: בניית מערכות סוכן יצירת כלי פרודוקטיביות למפתחים, וחילוץ נתונים, Claude Code ב-CD/CI פייפליין מחקר רב-סוכני, אינטגרציה של מובנים ממסמכים לא-מובנים.

### הפרופיל המבוקש

Claude. שמתכנן ומשגר אפליקציות פרודקשן עם (solution architect) המועמד האידיאלי הוא ארכיטקט פתרונות: רצויה לפחות חצי שנת ניסיון מעשי ב:

- Claude Agent SDK — hooks, אורקסטרציה רב-סוכנית, הענקת משימות לתת-סוכנים, אינטגרציית כלים — מחזור חיים
- Claude Code — CLAUDE.md, MCP, Agent Skills, שרתי, מצב תכנון (planning mode)
- Model Context Protocol (MCP) — backend-כלים ומשאבים לאינטגרציה אל ה
- הנדסת פרומפט — JSON schemas, few-shot, דוגמאות, תבניות לחילוץ נתונים
- עבודה עם מסמכים ארוכים, העברת הקשר רב-סוכני — (context window) חלון הקשר
- אוטומטי, יצירת בדיקות code review — CI/CD פייפליין
- human-in-the-loop, הסלמה ואמינות — טיפול בשגיאות

### פורמט המבחן

| פרמטר         | ערך                             |
|---------------|---------------------------------|
| סוג שאלה      | בחירה מרובה (1 נכון מתוך 4)     |
| ניקוד         | סולם 100–1000, ציון עובר 720    |
| עונש על ניחוש | אין (תענו על כל שאלה!)          |
| תרחישים       | מתוך 8 אפשריים (בחירה אקראית) 4 |

## תוכן המבחן: 5 דומיינים

| משקל | דומיין                          |
|------|---------------------------------|
| 27%  | ארכיטקטורת סוכנים ואורקסטרציה   |
| 18%  | MCP עיצוב כלים ואינטגרציית      |
| 20%  | זרימות עבודה Claude Code הגדרות |
| 20%  | הנדסת פרומפט ופלט מובנה         |
| 15%  | ניהול הקשר ואמינות              |

## תרחישי המבחן

### תרחיש 1: סוכן תמיכת לקוחות

הסוכן משתמש Claude Agent SDK-אתם בונים סוכן שמטפל בהחזרות, מחלוקות חיוב ובעיות חשבון תוך שימוש ב היעד הוא +80%. ( `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human` ) MCP בכלי פתרון במגע ראשון עם הסלמה מתאימה.

### Claude Code תרחיש 2: יצירת קוד עם

תיעוד. צריך לשלב פקודות, refactoring, debugging, להאצת פיתוח: יצירת קוד Claude Code-אתם משתמשים ב ולהבין מתי להשתמש במצב תכנון, CLAUDE.md מותאמות אישית והגדרות slash.

### תרחיש 3: מערכת מחקר רב-סוכנית

סוכן מתאם מאציל משימות לתת-סוכנים מתמחים: מחקר אינטרנטי, ניתוח מסמכים, סינתזה, ויצירת דוחות. המערכת חייבת להפיק דוחות מלאים עם ציטוטים.

### תרחיש 4: כלי פרודוקטיביות למפתחים

ולאוטומט משימות שגרתיות. נעשה שימוש boilerplate לא-מוכר, לייצר codebase הסוכן עוזר למהנדסים לחקור MCP ובשרתי (Read, Write, Bash, Grep, Glob) בכלים מובנים.

### 5 תרחיש: Claude Code ל-Continuous Integration

הפרומפטים חייבים. PR אוטומטי, יצירת בדיקות ופידבק על code review ל-CI/CD בפייפליין Claude Code שילוב false positives להיות מתוכננים למזער.

### תרחיש 6: חילוץ נתונים מובנים

ושומרת על דיוק גבוה. עליה לטפל JSON schema המערכת מחלצת מידע ממסמכים לא-מובנים, מאמתת פלט מול נכון במקרי קצה.

## שיחתי AI תרחיש 7: דפוסי ארכיטקטורת

אתם מתכננים מערכות שיחה רב-תורניות שמכסות ניהול חלון הקשר, התמדה של הוראות בין תורים, אסטרטגיות זיכרון, עיצוב כלים לביצוע בטוח, וטיפול בקלט משתמש דו-משמעי או סותר.

## סוכניים (תוכן חסר — עזרו לנו להשלים!) AI תרחיש 8: כלי

תרחיש זה דווח על ידי נבחנים אך טרם כוסה במדריך. אם נתקלתם בשאלות מהתרחיש הזה במבחן האמיתי, שתפו [ב-GitHub Issues](#) כדי שנוסיף כיסוי מלא. התרומה שלכם תעזור לכל מי שמתכונן למבחן

## תיעוד רשמי

| משאב                                        | כתובת                                                                                                                                                     |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude API — Messages                       | <a href="https://platform.claude.com/docs/en/api/messages">https://platform.claude.com/docs/en/api/messages</a>                                           |
| Claude API — Tool Use                       | <a href="https://platform.claude.com/docs/en/build-with-claude/tool-use">https://platform.claude.com/docs/en/build-with-claude/tool-use</a>               |
| Claude API — Message Batches                | <a href="https://platform.claude.com/docs/en/build-with-claude/message-batches">https://platform.claude.com/docs/en/build-with-claude/message-batches</a> |
| Claude Agent SDK — Overview                 | <a href="https://platform.claude.com/docs/en/agent-sdk/overview">https://platform.claude.com/docs/en/agent-sdk/overview</a>                               |
| Claude Agent SDK — Hooks                    | <a href="https://platform.claude.com/docs/en/agent-sdk/hooks">https://platform.claude.com/docs/en/agent-sdk/hooks</a>                                     |
| Claude Agent SDK — Subagents                | <a href="https://platform.claude.com/docs/en/agent-sdk/subagents">https://platform.claude.com/docs/en/agent-sdk/subagents</a>                             |
| Claude Agent SDK — Sessions                 | <a href="https://platform.claude.com/docs/en/agent-sdk/sessions">https://platform.claude.com/docs/en/agent-sdk/sessions</a>                               |
| Model Context Protocol (MCP)                | <a href="https://modelcontextprotocol.io/">https://modelcontextprotocol.io/</a>                                                                           |
| MCP — Tools                                 | <a href="https://modelcontextprotocol.io/docs/concepts/tools">https://modelcontextprotocol.io/docs/concepts/tools</a>                                     |
| MCP — Resources                             | <a href="https://modelcontextprotocol.io/docs/concepts/resources">https://modelcontextprotocol.io/docs/concepts/resources</a>                             |
| MCP — Servers                               | <a href="https://modelcontextprotocol.io/docs/concepts/servers">https://modelcontextprotocol.io/docs/concepts/servers</a>                                 |
| Claude Code — Documentation                 | <a href="https://code.claude.com/docs/en/overview">https://code.claude.com/docs/en/overview</a>                                                           |
| Claude Code — CLAUDE.md and Memory          | <a href="https://code.claude.com/docs/en/memory">https://code.claude.com/docs/en/memory</a>                                                               |
| Claude Code — Skills (incl. slash commands) | <a href="https://code.claude.com/docs/en/skills">https://code.claude.com/docs/en/skills</a>                                                               |
| Claude Code — Hooks                         | <a href="https://code.claude.com/docs/en/hooks">https://code.claude.com/docs/en/hooks</a>                                                                 |
| Claude Code — Sub-agents                    | <a href="https://code.claude.com/docs/en/sub-agents">https://code.claude.com/docs/en/sub-agents</a>                                                       |
| Claude Code — MCP Integration               | <a href="https://code.claude.com/docs/en/mcp">https://code.claude.com/docs/en/mcp</a>                                                                     |
| Claude Code — GitHub Actions CI/CD          | <a href="https://code.claude.com/docs/en/github-actions">https://code.claude.com/docs/en/github-actions</a>                                               |

|                                               |                                                                                                                                                                                   |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude Code — GitLab CI/CD                    | <a href="https://code.claude.com/docs/en/gitlab-ci-cd">https://code.claude.com/docs/en/gitlab-ci-cd</a>                                                                           |
| Claude Code — Headless (non-interactive mode) | <a href="https://code.claude.com/docs/en/headless">https://code.claude.com/docs/en/headless</a>                                                                                   |
| Prompt Engineering Guide                      | <a href="https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview">https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview</a> |
| Extended Thinking                             | <a href="https://platform.claude.com/docs/en/build-with-claude/extended-thinking">https://platform.claude.com/docs/en/build-with-claude/extended-thinking</a>                     |
| Anthropic Cookbook (code examples)            | <a href="https://github.com/anthropics/anthropic-cookbook">https://github.com/anthropics/anthropic-cookbook</a>                                                                   |

## יסודות תיאורטיים: חלק

חלק זה מכסה את כל התיאוריה שצריך לעבור בהצלחה את המבחן. החומר מאורגן לפי טכנולוגיות ומושגים ולא לפי דומינים — כך בונים הבנה עמוקה יותר של כל נושא.

## יסודות אינטראקציה עם המודל — Claude API: פרק 1

תיעוד: [Messages API](#) | [Prompt Engineering](#)

### 1.1 API מבנה בקשת

כוללת Claude Messages API פועל על פי מודל בקשה-תגובה. כל בקשה ל Claude API-

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    { "role": "user", "content": "Hi!" },
    { "role": "assistant", "content": "Hello!" },
    { "role": "user", "content": "How are you?" }
  ],
  "tools": [...],
  "tool_choice": { "type": "auto" }
}
```

שדות מפתח:

- `model` — בחירת מודל ( `claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5` )
- `max_tokens` — מספר טוקנים מקסימלי בתגובה
- `system` — system prompt (מגדיר התנהגות המודל)

- `messages` — היסטוריית שיחה (חובה לשלוח את ההיסטוריה המלאה כדי לשמור על קוהרנטיות)
- `tools` — הגדרות הכלים הזמינים
- `tool_choice` — אסטרטגיית בחירת כלי

## 1.2 תפקידי הודעות (Message Roles)

משתמש בשלושה תפקידים `messages`-מערך ה:

- `user` — הודעות משתמש
- `assistant` — תגובות המודל (כלולות בשליחת היסטוריה)
- `tool` — התפקיד אינו מוגדר במפורש; זה מופיע כ) תוצאות קריאה לכלי — `tool_result` content block

בין בקשות — כל state חייבים לשלוח את כל היסטוריית השיחה. המודל לא שומר API חשוב במיוחד: בכל בקשת קריאה עצמאית.

## 1.3 השדה `stop_reason` בתגובה

שמציין מדוע המודל הפסיק לייצר `stop_reason` כוללת Claude API תגובת

| ערך             | תיאור                  | פעולה                                 |
|-----------------|------------------------|---------------------------------------|
| "end_turn"      | המודל סיים את תגובתו   | הציגו תוצאה למשתמש                    |
| "tool_use"      | המודל רוצה לקרוא לכלי  | הריצו את הכלי והחזירו תוצאה           |
| "max_tokens"    | הגעתם לגבול טוקנים     | התגובה קטומה; ייתכן שצריך להגדיל גבול |
| "stop_sequence" | stop sequence-נתקלתם ב | טפלו לפי לוגיקת האפליקציה             |

הם החשובים ביותר — הם שולטים בלולאת הסוכן "end\_turn" ו- "tool\_use" במערכות סוכן

## 1.4 System Prompt

הוא הוראה מיוחדת שמגדירה הקשר וכללי התנהגות. הוא system prompt-ה:

- `system` מועבר בנפרד בשדה `messages`; לא חלק ממערך ה
- בעדיפות גבוהה יותר מהודעות משתמש
- נטען פעם אחת וחל לאורך כל השיחה
- משמש להגדרת תפקיד, מגבלות ופורמט פלט

יכול ליצור אסוציאציות לא-רצויות לכלים. למשל, הוראה כמו "תאמת תמיד system prompt חשוב למבחן: ניסוח של גם כשזה לא נחוץ, `get_customer`-את הלקוח" עלולה לגרום למודל להשתמש יותר מדי ב

## 1.5 חלון הקשר (Context Window)

חלון ההקשר הוא הכמות הכוללת של טקסט (בטוקנים) שהמודל יכול לעבד בבת אחת. הוא כולל

- system prompt

- היסטוריית ההודעות המלאה
- הגדרות כלים
- תוצאות כלים

### בעיות מפתח של חלון ההקשר

1. מודלים מעבדים מידע באמינות בהתחלה ובסוף של קלט ארוך, אבל עלולים לפספס **lost-in-the-middle אפקט**. פרטים באמצע. מיתון: מקמו מידע קריטי קרוב לתחילה או לסוף.
2. **הצטברות תוצאות כלי**: כל קריאה לכלי מוסיפה פלט להקשר. אם כלי מחזיר +40 שדות אבל רק 5 רלוונטיים — רוב ההקשר מבוזז.
3. **סיכום פרוגרסיבי**: בעת דחיסת היסטוריה, ערכים מספריים, אחוזים ותאריכים אובדים לעיתים והופכים מעורפלים ("בערך", "בקירוב", "כמה").

## 2 Tools - tool\_use: פרק 2

תיעוד: Tool Use

### 2.1 מה זה tool\_use

לקרוא לפונקציות חיצוניות. המודל לא מריץ קוד בעצמו — הוא מייצר Claude-הוא מנגנון שמאפשר ל tool\_use בקשת קריאה מובנית; הקוד שלכם מריץ אותה ומחזיר את התוצאה.

### 2.2 הגדרת כלי

JSON schema כל כלי מוגדר באמצעות:

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile, including name, email, order history, and account status. Use this tool BEFORE lookup_order to verify the customer's identity. Accepts an email (format: user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

היבטים קריטיים של תיאור הכלי:

- בוחר כלים על בסיס התיאור שלהם. תיאורים מינימליים ("מאחזר LLM. התיאור הוא מנגנון הבחירה העיקרי. מידע לקוח") מובילים לטעויות כשכלים חופפים.
- כללו בתיאור:
  - מה הכלי עושה ומחזיר
  - פורמטי קלט וערכים לדוגמה
  - מקרי קצה ומגבלות
  - מתי להשתמש בכלי הזה לעומת חלופות דומות
- יש תיאורים `analyze_document` ו-`analyze_content` -הימנעו מתיאורים זהים או חופפים בין כלים. אם ל. כמעט זהים — המודל יבלבל ביניהם.
- עם MCP על פני כלי (Read, Grep) סוכנים עשויים להעדיף כלים מובנים: **MCP כלים מובנים מול כלי**. הדגישו יתרונות קונקרטיים, נתונים ייחודיים או הקשר: MCP פונקציונליות דומה. כדי למנוע — חזקו את תיאורי כלי שכלים מובנים לא יכולים לספק.

## 2.3 הפרמטר `tool_choice`

שולט באופן שבו המודל בוחר כלים `tool_choice`:

| מתי להשתמש                         | התנהגות                                  | ערך                                                       |
|------------------------------------|------------------------------------------|-----------------------------------------------------------|
| ברירת מחדל לרוב המקרים             | המודל מחליט אם לקרוא לכלי או לענות בטקסט | <code>{"type": "auto"}</code>                             |
| כשצריך פלט מובנה מובטח             | המודל <b>חייב</b> לקרוא לאיזשהו כלי      | <code>{"type": "any"}</code>                              |
| כשצריך לכפות צעד ראשון / סדר הפעלה | המודל <b>חייב</b> לקרוא לכלי ספציפי      | <code>{"type": "tool", "name": "extract_metadata"}</code> |

### תרחישים חשובים:

- מספר כלי חילוץ → המודל בוחר את המתאים ביותר אבל אתם עדיין מקבלים פלט + `tool_choice: "any"` מובנה
- (לפני העשרה `extract_metadata` למשל) בחירה כפויה → כשחייבים להבטיח פעולה ראשונה מסוימת

## 2.4 JSON Schemas לפלט מובנה

הוא Claude-הוא הדרך **האמינה ביותר** לקבל פלט מובנה מ JSON schemas עם `tool_use` -שימוש ב

- תקין מבחינה תחבירית (אין סוגריים חסרים, פסיקים מיותרים) JSON מבטיח
- (נוכחים `required` שדות) אוכף את המבנה הנדרש
- לא** מבטיח נכונות סמנטית (ערכים עדיין יכולים להיות שגויים)

**עיצוב סכמה — עקרונות מפתח**

```
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "Details if category = 'other' or 'unclear'"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "Null if the information was not found in the source"
    }
  },
  "required": ["category", "severity"]
}
```

### כללי עיצוב סכמה

1. **Required מול optional:** דוחפים את המודל required רק אם המידע תמיד זמין. שדות required-סמנו שדות כ: **Required**. לבדוד ערכים כשהמידע חסר.
2. **nullable שדות:** למידע שעלול להיות חסר. המודל יכול להחזיר `"type": ["string", "null"]` -השתמשו ב: `null` במקום להזות `null`.
3. **Enums עם "other":** שדה פירוט כדי לא לאבד נתונים מחוץ לקטגוריות שהגדרתם + `"other"` הוסיפו: `"other"` עם `Enum` מראש.
4. **Enum "unclear":** כן הוא טוב יותר `"unclear"` — למקרים שבהם המודל לא יכול לבחור קטגוריה בביטחון: `"unclear"` Enum. מקטגוריה שגויה.

## שגיאות תחביריות מול סמנטיות 2.5

| סוג שגיאה | דוגמה                                     | מיתון                                              |
|-----------|-------------------------------------------|----------------------------------------------------|
| תחבירית   | לא תקין, סוג שדה שגוי JSON                | (מבטל) JSON schema עם <code>tool_use</code>        |
| סמנטית    | סכומים לא מסתכמים, ערך בשדה לא נכון, הזיה | retry-with-feedback, self-correction, בדיקות אימות |



# בניית מערכות סוכן — Claude Agent SDK: פרק 3

תיעוד: [Agent SDK](#) | [Hooks](#) | [Subagents](#) | [Sessions](#)

## 3.1 מהי לולאת סוכן (Agentic Loop)

לולאת הסוכן היא הדפוס הליבתי לביצוע משימות אוטונומיות. המודל לא רק עונה — הוא מבצע רצף פעולות:

- עם כלים Claude-שלחו בקשה ל.
- קבלו תגובה.
- בדקו `stop_reason`:
  - הריצו את הכלי, צרפו תוצאה להיסטוריה, חזרו לצעד 1 - `"tool_use"`
  - המשימה הושלמה, הציגו תוצאה למשתמש - `"end_turn"`
- חזרו עד להשלמה.

מחליט באיזה כלי לקרוא בא הלאה על בסיס הקשר ותוצאות קודמות. שונה Claude **model-driven**: זוהי גישה מעצי החלטה מקודדים-קשיחים שבהם הרצף קבוע מראש.

אנטי-פטרנים (להימנע):

- "Task completed" כדי לזהות סיום assistant-ניתוח טקסט של ה.
- כתנאי עצירה ראשי (`max_iterations=5`) שימוש בגבול איטרציות שרירותי.
- ייצר תוכן טקסטואלי כאות סיום assistant-בדיקה אם ה.

. `stop_reason == "end_turn"` **גישה נכונה**: הסימן האמין היחיד לסיום הוא

## 3.2 הגדרת AgentDefinition

Claude Agent SDK-הוא אובייקט הגדרת הסוכן ב `AgentDefinition`:

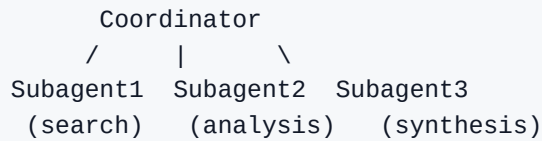
```
agent = AgentDefinition(
    name="customer_support",
    description="Handles customer requests for returns and order issues",
    system_prompt="You are a customer support agent...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
    # For a coordinator:
    # allowed_tools=["Task", "get_customer", ...]
)
```

פרמטרים מרכזיים:

- `name` / `description` — זיהוי ותיאור הסוכן
- `system_prompt` — system prompt עם הוראות
- `allowed_tools` — רשימת הכלים המותרים (עקרון הרשאה מינימלית)

### 3.3 Hub-and-Spoke: מתאם ותת-סוכנים

hub-and-spoke: ארכיטקטורה רב-סוכנית בדרך כלל בנויה כטופולוגיית



**המתאם אחראי על:**

- פירוק המשימה לתת-משימות
- החלטה אילו תת-סוכנים נדרשים (בחירה דינמית)
- האצלת עבודה לתת-סוכנים
- צבירה ואימות תוצאות
- retries-טיפול בשגיאות ו
- העברת תוצאות למשתמש

**עקרון קריטי: לתת-סוכנים יש הקשר מבודד.**

- תת-סוכנים אינם יורשים אוטומטית את היסטוריית השיחה של המתאם
- כל ההקשר הנדרש חייב להיות מועבר במפורש בפרומפט של תת-הסוכן
- תת-סוכנים לא חולקים זיכרון בין קריאות
- כל התקשורת זורמת דרך המתאם (לצורך תצפיתיות ובקרת שגיאות)

### 3.4 ליצירת תת-סוכנים Task הכלי

Task: תת-סוכנים נוצרים דרך הכלי

```
# The coordinator's allowedTools must include "Task"
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

**העברת הקשר מפורשת היא חובה:**

```
# Bad: the subagent has no context
Task: "Analyze the document"

# Good: full context in the prompt
Task: "Analyze the following document.
Document: [full document text]
Prior search results: [web search results]
Output format requirements: [schema]"
```

ים בתגובה אחת — תת-הסוכנים רצים במקביל- Task הפעלה מקבילית: מתאם יכול לקרוא למספר

```
# One coordinator response contains:
Task 1: "Search for articles about X"
Task 2: "Analyze document Y"
Task 3: "Search for articles about Z"
# All three run concurrently
```

### 3.5 Hooks ב-Agent SDK

מאפשרים ירוט וטרנספורמציה בנקודות ספציפיות במחזור החיים של הסוכן Hooks.

מיירט תוצאת כלי לפני שהיא מועברת למודל **PostToolUse**:

```
# Example: normalize date formats from different MCP tools
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Convert Unix timestamp -> ISO 8601
    # Convert "Mar 5, 2025" -> "2025-03-05"
    return normalized_result
```

ליירוט קריאות יוצאות חוסם פעולות שמפרות מדיניות **Hook**:

```
# Example: block refunds above $500
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

מול הוראות בפרומפט **hooks**: הבדל מרכזי

| תכונה      | Hooks                                         | הוראות בפרומפט               |
|------------|-----------------------------------------------|------------------------------|
| הבטחה      | דטרמיניסטית (100%)                            | הסתברותית (<90%, לא 100%)    |
| מתי להשתמש | compliance, כללי עסק קריטיים, פעולות פיננסיות | העדפות כלליות, המלצות, פורמט |
| דוגמה      | חסום החזרים < 500\$                           | "נסה לפתור לפני הסלמה"       |

לא בפרומפטים, **hooks-כלל**: כששגיאה יוצרת תוצאות פיננסיות, משפטיות או בטיחותיות — השתמשו ב

## פרק 4: Model Context Protocol (MCP)

## 4.1 מה זה MCP

Model Context Protocol (MCP) הוא פרוטוקול פתוח לחיבור מערכות חיצוניות ל Claude. MCP סוגי שלושה מגדיר: משאבים עיקריים:

1. **Tools** — (הרצת פקודות, API קריאות, CRUD) פונקציות שהסוכן יכול לקרוא להן כדי לבצע פעולות
2. **Resources** — (קטלוגי תוכן, DB תיעוד, סכמות) נתונים שהסוכן יכול לקרוא להקשר
3. **Prompts** — תבניות פרומפט מוגדרות מראש למשימות נפוצות

## 4.2 שרתי MCP

MCP כשמתחברים לשרת tools/resources ומספק MCP הוא תהליך שמממש את פרוטוקול MCP שרת:

- כל הכלים מתגלים אוטומטית
- כלים מכל השרתים המחוברים זמינים בו-זמנית
- תיאורי הכלים קובעים איך המודל ישתמש בהם

## 4.3 הגדרת שרתי MCP

לשימוש צוותי — ( `.mcp.json` ) הגדרת פרויקט:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

נקודות מפתח:

- VCS-נשמר בשורש הפרויקט ומנוהל ב `.mcp.json`
- משמשים לסודות — הטוקנים עצמם לא מוקמטים ( `${GITHUB_TOKEN}` ) משתני סביבה
- זמין לכל תורמי הפרויקט

לשרתים אישיים/ניסיוניים — ( `~/.claude.json` ) הגדרת משתמש:

- של המשתמש home directory-נשמר ב

- VCS לא משותף דרך
- מתאים לניסיונות אישיים ובדיקות

#### בחירת שרתים:

- קהילתיים קיימים MCP העדיפו שרתי — (Jira, GitHub, Slack) לאינטגרציות סטנדרטיות
- בנו שרתים משלכם רק לזרימות עבודה ייחודיות וצוותיות

## 4.4 הדגל MCP ב-`isError`

בתגובה. זה מסמן לסוכן שהקריאה נכשלה `isError: true` -נתקל בשגיאה, הוא משתמש ב MCP כשכלי

#### שגיאה מובנית (טוב):

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable. Timeout while calling the orders API.",
    "attempted_query": "order_id=12345",
    "partial_results": null
  }
}
```

#### שגיאה גנרית (אנטי-פטרן):

```
{
  "isError": true,
  "content": "Operation failed"
}
```

שגיאה גנרית לא נותנת לסוכן שום מידע להחלטה — האם לנסות שוב, לשנות שאילתה או להסלים?

## 4.5 MCP Resources

הם נתונים שסוכן יכול לבקש כדי לקבל הקשר בלי לבצע פעולות Resources:

- קטלוגי תוכן (למשל רשימת כל משימות הפרויקט, ניווט היררכי)
- סכמות מסד נתונים (הבנת מבנה הנתונים)
- (מדריכים פנימיים, API references) תיעוד
- issue/task סיכומי

מספק "מפה" Resource. הסוכן לא צריך קריאות כלי חקרניות כדי להבין אילו נתונים קיימים **Resources יתרון**. מיידית.

# הגדרות וזרימות עבודה — Claude Code: פרק 5

תיעוד: [Claude Code](#) | [Memory / CLAUDE.md](#) | [Skills](#) | [MCP](#) | [Hooks](#) | [Sub-agents](#) | [GitHub Actions](#) | [Headless](#)

## 5.1 CLAUDE.md ההיררכיה של

יש היררכיה תלת-שכבתית. Claude Code הוא קובץ/קבצי ההוראות ל CLAUDE.md:

1. רמת משתמש: `~/.claude/CLAUDE.md`
  - חל רק על המשתמש הזה
  - VCS לא משותף דרך
  - העדפות אישיות וסגנון עבודה
2. בשורש CLAUDE.md או `.claude/CLAUDE.md`: רמת פרויקט
  - חל על כל תורמי הפרויקט
  - VCS מנוהל דרך
  - סטנדרטי קוד, סטנדרטי בדיקה, החלטות ארכיטקטוניות
3. בתת-תיקיות CLAUDE.md: רמת תיקייה
  - חל בעת עבודה עם קבצים בתיקייה
  - codebase-מוסכמות ספציפיות לחלק זה של ה

(רמת משתמש) `~/.claude/CLAUDE.md` - **טעות נפוצה:** חבר צוות חדש לא מקבל הוראות פרויקט כי הן הושמו ב `.claude/CLAUDE.md` (רמת פרויקט) - במקום ב

## 5.2 (יבוא קבצים) @path תחביר

מה שהופך את ההגדרה למודולרית, @path יכול להתייחס לקבצים חיצוניים באמצעות CLAUDE.md:

```
# Project CLAUDE.md
```

```
Coding standards are described in @./standards/coding-style.md
Test requirements are in @./standards/testing-requirements.md
Project overview is in @README.md and dependencies are in @package.json
```

כללי @path:

- השתמשו ב-@ מיד לפני נתיב הקובץ (בלי רווח)
- נתמכים נתיבים יחסיים ומוחלטים
- נתיבים יחסיים נפתרים יחסית לקובץ שמכיל את היבוא
- עומק קינון מקסימלי של יבוא הוא 5

זה מונע כפילויות ומאפשר לכל חבילה לכלול רק סטנדרטי רלוונטיים.

## 5.3 התיקיה `.claude/rules/`

מונוליטי, לארגון כללים לפי נושא CLAUDE.md היא חלופה ל `.claude/rules/`.

```
.claude/rules/
testing.md          -- testing conventions
api-conventions.md  -- API conventions
deployment.md       -- deployment rules
react-patterns.md   -- React patterns
```

לטעינה מותנית `paths` עם `YAML frontmatter`: **תכונה מרכזית**

```
---
paths: ["src/api/**/*.js"]
---
```

For API files, use `async/await` with explicit error handling.  
Each endpoint must return a standard response wrapper.

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---
```

Tests must use `describe/it` blocks.  
Use data factories instead of hardcoding.  
Do not mock the database—use a test database.

**איך זה עובד**

- עורך קובץ שמתאים לתבנית Claude Code כלל נטען רק כש `paths`
- זה חוסך הקשר וטוקנים — כללים לא-רלוונטיים לא נטענים
- מאפשרות להחיל מוסכמות לפי סוג קובץ ללא תלות במיקום (אידיאלי לבדיקות פזורות) `glob` תבניות

**ברמת תיקייה CLAUDE.md לעומת `paths` עם `.claude/rules/` -מתי להשתמש ב**

- `(tests, migrations)` כשמוסכמות חלות על קבצים פזורים בהרבה תיקיות — עם `paths` `.claude/rules/`
- ברמת תיקייה — כשמוסכמות קשורות לתיקיה ספציפית ולא נחוצות במקום אחר CLAUDE.md

## 5.4 Custom Slash Commands ו-Skills

( `.claude/commands/` ) פקודות מותאמות אישית, Claude Code **הערה**: בגרסה הנוכחית של מדריך המבחן `/name` שני הפורמטים יוצרים פקודות ( `.claude/skills/` ) `skills` מאוחדות עם הפורמט הזה עדיין נתמך — `.claude/commands/` -מתייחס ל

`/name` : הן תבניות פרומפט הניתנות לשימוש חוזר שמופעלות דרך `Slash commands`

**פורמט (legacy, נתמך) `.claude/commands/` :**

```
.claude/commands/  
review.md          -- /review -- standard code review  
test-gen.md        -- /test-gen -- test generation
```

**פורמט** `.claude/skills/` (נוכחי):

```
.claude/skills/  
review/SKILL.md    -- /review -- with frontmatter configuration  
test-gen/SKILL.md
```

**פקודות פרויקט** (`.claude/commands/` או `.claude/skills/`):

- של הריפו clone וזמינות לכולם בעת VCS-נשמרות ב
- מבטיחות זרימות עבודה עקביות בכל הצוות

**פקודות משתמש** (`~/.claude/commands/` או `~/.claude/skills/`):

- VCS פקודות אישיות שלא משותפות דרך
- לזרימות עבודה אינדיבידואליות

## 5.5 Skills — `.claude/skills/`

Skills של frontmatter הן פקודות מתקדמות שמוגדרות דרך SKILL.md:

```
---  
context: fork  
allowed-tools: ["Read", "Grep", "Glob"]  
argument-hint: "Path to the directory to analyze"  
---  
  
Analyze the code structure in the specified directory.  
Output a report on dependencies and architectural patterns.
```

**Frontmatter פרמטרי:**

| פרמטר                      | תיאור                                                                        |
|----------------------------|------------------------------------------------------------------------------|
| <code>context: fork</code> | לא מזהם את הסשן הראשי verbose בתת-סוכן מבודד. פלט skill-מריץ את ה            |
| <code>allowed-tools</code> | (לא יכול למחוק קבצים אם לא הותר skill-אבטחה — למשל ה) מגביל אילו כלים זמינים |
| <code>argument-hint</code> | רמז שמבקש ארגומנט בעת הפעלה ללא פרמטרים                                      |

**CLAUDE.md לעומת skill מתי:**

- Skill** — (ניתוח, יצירה, review) למשימה ספציפית on-demand הפעלה —
- CLAUDE.md** — סטנדרטי ומוסכמות כלליים שטעונים תמיד

**Skills אישיים** (`~/.claude/skills/`):

- צרו ואריאנטים אישיים בשמות שונים כדי לא להשפיע על חברי הצוות



## לעומת ביצוע ישיר (Planning Mode) מצב תכנון 5.6

### מצב תכנון:

- המודל רק חוקר ומתכנן; לא מבצע שינויים
- codebase-כדי לחקור את ה Read, Grep, Glob-משתמש ב
- מייצר תוכנית מימוש שהמשתמש מאשר
- חקירה בטוחה בלי תופעות לוואי

### מתי להשתמש במצב תכנון:

- שינויים גדולים (עשרות קבצים)
- (איך להגדיר גבולות: microservices) מספר גישות סבירות
- (איזה מבנה? framework איזה) החלטות ארכיטקטוניות
- לא מוכר (חייבים להבין לפני שמשנים) codebase
- מיגרציות ספרייה שמשפיעות על +45 קבצים

### מתי להשתמש בביצוע ישיר:

- ברור stack trace תיקון בקובץ יחיד עם
- בודדת validation הוספת בדיקת
- שינויים מובנים היטב ולא דו-משמעיים

### גישה משולבת:

1. מצב תכנון לחקירה ועיצוב
2. המשתמש מאשר את התוכנית
3. ביצוע ישיר ליישום התוכנית המאושרת

### codebase-תת-סוכן מתמחה לחקירת ה Explore subagent:

- מההקשר הראשי verbose מבודד פלט
- מחזיר רק סיכום
- מונע אזילת חלון הקשר במשימות רב-שלביות

## 5.7 הפקודה /compact

היא פקודה מובנית לדחיסת הקשר /compact:

- מסכמת היסטוריה קודמת כדי לפנות חלון הקשר
- verbose משמשת בסשני חקירה ארוכים שבהם ההקשר מתמלא בפלט כלי
- סיכון: ערכים מספריים מדויקים, תאריכים ופרטים ספציפיים עלולים ללכת לאיבוד בסיכום

## 5.8 הפקודה /memory

היא פקודה מובנית לניהול זיכרון בין סשנים /memory:

- לעריכה, מאפשרת לשמור הערות, העדפות והקשר CLAUDE.md פותחת את הקובץ
- מידע נמשך בין סשנים ונטען אוטומטית בהפעלה

- שימושית לשמירת מוסכמות פרויקט, העדפות משתמש, פקודות בשימוש תכוף, והקשר עבודה נוכחי
- חלופה להסבר חוזר של אותן הוראות בכל ששן

## 5.9 Claude Code CLI ל-CI/CD

הדגל `-p` (או `--print`):

```
claude -p "Analyze this pull request for security issues"
```

- יוצא, stdout-מצב לא-אינטראקטיבי: מעבד את הפרומפט, מדפיס ל
- לא מחכה לקלט משתמש
- CI/CD בפייפליין Claude הדרך הנכונה היחידה להריץ

ל-CI-פלט מובנה

```
claude -p "Review this PR" --output-format json --json-schema  
'{"type":"object",...}'
```

- `--output-format json` — JSON-פלט ב
- `--json-schema` — אימות פלט מול סכמה
- אוטומטית PR-על ה inline ניתן לפרסר את התוצאה כדי לפרסם הערות

שייצר קוד הוא לרוב פחות אפקטיבי בסקירתו (המודל שומר את הקשר Claude **בידוד הקשר של ששן**: אותו ששן review-עצמאי ל instance-השיקול שלו ופחות סביר שיאתגר את ההחלטות של עצמו). השתמשו ב

הקודמות בהקשר והנחו review-אחרי קומיטים חדשים, כללו את תוצאות ה re-review **מניעת הערות כפולות**: בעת לדווח רק על בעיות חדשות או לא-פתורות Claude את

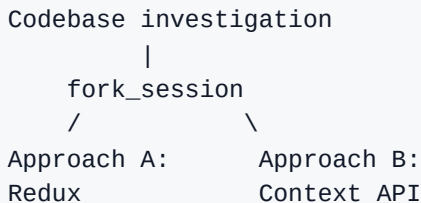
## 5.10 `fork_session` וניהול ששנים

ממשיך ששן בעל שם `--resume <session-name>`:

```
claude --resume investigation-auth-bug
```

- ממשיך שיחה קודמת עם הקשר שמור
- שימושי לחקירות ארוכות שמתפרסות על מספר ששנים
- סיכון: אם קבצים השתנו מאז הסשן הקודם, תוצאות הכלים עלולות להיות מיושנות

יוצר סניף עצמאי מהקשר משותף `fork_session`:



- יורשים הקשר עד נקודת ההסתעפות forks-שני ה
- אחרי כן הם מתפצלים עצמאית
- שימושי להשוואת גישות או בדיקת אסטרטגיות

**מתי להתחיל סשן חדש במקום להמשיך:**

- תוצאות הכלים מיושנות (קבצים השתנו)
- עבר הרבה זמן והקשר ירד באיכותו
- עדיף להתחיל מחדש עם "הנה סיכום קצר של מה שמצאנו: ..." במקום להמשיך עם נתוני כלי ישנים

## פרק 6: הנדסת פרומפט — טכניקות מתקדמות

תיעוד: [Prompt Engineering](#) | [Anthropic Cookbook](#)

### 6.1 Few-shot Prompting

הוא הכללה של 2–4 דוגמאות קלט/פלט בפרומפט כדי להדגים את ההתנהגות הצפויה Few-shot prompting.

**אפקטיבי יותר מתיאורים טקסטואליים few-shot למה:**

- הוראה מעורפלת כמו "תהיה מדויק יותר" יכולה להתפרש בהרבה דרכים
- דוגמה מראה באופן חד-משמעי את הפורמט הצפוי ולוגיקת ההחלטה
- המודל מכליל את הדפוס למקרים חדשים (הוא לא רק חוזר על הדוגמאות)

**ומתי להשתמש בהן few-shot סוגי דוגמאות:**

1. **דוגמאות לתרחישים דו-משמעיים:**

```
Request: "My order is broken"
Action: Call get_customer -> lookup_order -> check status.
Rationale: "broken" may mean a damaged item; you need order details.
```

```
Request: "Get me a manager"
Action: Immediately call escalate_to_human.
Rationale: The customer explicitly requests a human. Do not attempt to solve autonomously.
```

2. **דוגמאות לפורמט פלט:**

Finding example:

```
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

### 3. דוגמאות להפרדת קוד מקובל מקוד בעייתי:

```
// Acceptable (do not flag):
const items = data.filter(x => x.active);

// Problem (flag):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

### 4. דוגמאות לחילוף מפורמטי מסמכים שונים:

Document with inline citations:

"As shown in the study (Smith, 2023), the rate is 42%."

-> {"value": "42%", "source": "Smith, 2023", "type": "inline\_citation"}

Document with bibliography references:

"The rate is 42%. [1]"

-> {"value": "42%", "source": "reference\_1", "type": "bibliography"}

### 5. דוגמאות למדידות לא-פורמליות:

Text: "about two handfuls of rice"

-> {"amount": "~100g", "original\_text": "two handfuls", "precision": "approximate"}

Text: "a pinch of salt"

-> {"amount": "~1g", "original\_text": "a pinch", "precision": "approximate"}

אפקטיבי במיוחד לחילוף יחידות מדידה לא-פורמליות ולא-סטנדרטיות, שמגוונות מדי לכללים בלבד Few-shot.

מחמירים לפלט מובנה, הוסיפו כללי JSON schemas-כללי נורמליזציה פורמט בפרומפט: בעת שימוש ב נורמליזציה בפרומפט:

Normalization:

- Dates: always ISO 8601 (YYYY-MM-DD); "yesterday" -> compute an absolute date
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency": "USD"}
- Percentages: decimal fraction; "half" -> 0.5

תקין תחבירית אבל הערכים לא עקביים JSON-זה מונע שגיאות סמנטיות שבהן ה

## קריטריונים מפורשים לעומת הוראות מעורפלות 6.2

רע (מעורפל):

Check code comments for accuracy.  
Be conservative—report only high-confidence findings.

### טוב (קריטריונים מפורשים):

Flag a comment as problematic ONLY if:

1. The comment describes behavior that CONTRADICTS the actual code behavior
2. The comment references a non-existent function or variable
3. A TODO/FIXME comment refers to a bug that has already been fixed in code

Do NOT flag:

- Comments that are merely stylistically outdated
- Comments with minor wording inaccuracies
- Missing comments (that is a separate category)

### הגדירו קריטריוני חומרה עם דוגמאות:

CRITICAL: Runtime failure for users  
Example: NullPointerException while processing a payment

HIGH: Security vulnerability  
Example: SQL injection, XSS, missing authorization checks

MEDIUM: Logic bug without immediate impact  
Example: Wrong sorting, off-by-one error

LOW: Code quality  
Example: Duplication, suboptimal algorithm for small data

## 6.3 Prompt Chaining

מפרק משימה מורכבת לרצף שלבים ממוקדים

Step 1: Analyze auth.ts (local issues only)  
-> Output: list of issues in auth.ts

Step 2: Analyze database.ts (local issues only)  
-> Output: list of issues in database.ts

Step 3: Integration pass (cross-file dependencies)  
-> Output: issues at module boundaries

### למה זה חשוב:

- מונע דילול תשומת לב — כשהמודל מקבל יותר מדי קבצים בבת אחת, הוא עלול לפספס באגים בחלקם ולתת הערות שטחיות על אחרים
- מבטיח איכות ניתוח עקבית לכל קובץ
- מאפשר ניתוח נפרד של אינטראקציות בין-קבצים

dynamic decomposition לעומת prompt chaining מתי

- **Prompt chaining** — משימות צפויות וחזרתיות (code review, קבצים, מיגרציות)
- **Dynamic decomposition** — חקירות פתוחות שבהן תת-המשימות מתבהרות רק במהלך הביצוע

## 6.4 "Interview"-דפוס ה

שואל שאלות הבהרה Claude, לפני יישום פתרון:

Claude: "Before implementing caching for the API, a few questions:  
 1. Which cache invalidation strategy do you prefer—TTL or event-based?  
 2. Is stale data acceptable when the cache is unavailable?  
 3. Should caching be per-user or global?  
 4. What is the expected data volume to cache?"

**מתי זה שימושי:**

- (מערכות משפטיות, fintech, healthcare) דומיין לא מוכר
- (מצבי כשל, cache אסטרטגיות) משימות עם השלכות לא-מובנות
- מספר גישות סבירות שבחירת הטובה תלויה בהקשר

## 6.5 Validation I-Retry-with-Feedback

כשנתונים מחולצים נכשלים באימות:

Step 1: Extract data from the document  
 Step 2: Validate (Pydantic, JSON Schema, business rules)  
 Step 3: If there's an error—retry with context:  
 - The original document  
 - The previous (incorrect) extraction  
 - The specific error: "Field 'total' = 150, but sum(line\_items) = 145. Re-check values."

**יהיה אפקטיבי retry מתי:**

- שגיאות פורמט (תאריך בפורמט שגוי)
- שגיאות מבניות (שדה ממוקם במקום הלא נכון)
- אי-עקביות אריתמטיות (המודל יכול לבדוק מחדש)

**לא יעזור retry מתי:**

- המידע לא קיים במסמך המקור
- ההקשר הנדרש חיצוני (הנתונים במסמך אחר שלא סופק)

**לאימות נתונים מבוסס-סכמה.** למבחן, הנקודות החשובות Python היא ספריית Pydantic: **ככלי אימות Pydantic**

- Claude מ-JSON נבדקות בקוד אחרי קבלת enum מגבלות, requiredness, **אימות מבני:** סוגים
- **אימות סמנטי:** validators עסקית (לוגיקה אכופים מותאמים לכולם); start\_date < end\_date)
- עם הקשר השגיאה Claude-בנו הודעת שגיאה וקראו שוב ל Pydantic, עם כשל אימות: **validate-retry לולאות**

- מספקים מקור אמת, `tool_use` ל-JSON Schema יכולים לייצר Pydantic מודלים של **JSON Schema יצירת** יחיד

## 6.6 Self-correction

דפוס לזיהוי סתירות פנימיות:

```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

מאפשר לטפל באי-`conflict_detected`, המודל מחלץ גם את הערך המוצהר וגם ערך מחושב — אם הם שונים התאמה.

# 7 Message Batches API: פרק 7

תיעוד: [Message Batches](#)

## 7.1 סקירה

של בקשות לעיבוד אסינכרוני batch מאפשר לכם להגיש Message Batches API ה-

| תכונה                   | ערך                                               |
|-------------------------|---------------------------------------------------|
| חיסכון                  | לעומת קריאות סינכרוניות 50%                       |
| חלון עיבוד              | (לאיחור SLA אין הבטחת) עד 24 שעות                 |
| Multi-turn tool calling | לא נתמך (בקשה אחת = תגובה אחת)                    |
| קישור                   | לקישור בין בקשה לתגובה <code>custom_id</code> שדה |

## 7.2 Synchronous API לעומת Batch API מתי

| משימה               | API         | למה                          |
|---------------------|-------------|------------------------------|
| merge לפני PR בדיקת | Synchronous | המפתח מחכה; 24 שעות לא מקובל |

|                         |             |                                   |
|-------------------------|-------------|-----------------------------------|
| לילה tech-debt דוח      | Batch       | התוצאה נדרשת לבוקר; חיסכון של 50% |
| ביקורת אבטחה שבועית     | Batch       | לא דחוף; חיסכון של 50%            |
| אינטראקטיבי Code review | Synchronous | נדרשת תגובה מיידי                 |
| עיבוד 10,000 מסמכים     | Batch       | עיבוד מסיבי; החיסכון משמעותי      |

### שימוש ב custom\_id-7.3

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract data from: ..."}]
  }
}
```

מאפשר לכם custom\_id:

- לקשר את התוצאה למסמך המקורי
- בכשל, להגיש מחדש רק את המסמכים שנכשלו
- להימנע מעיבוד חוזר של מסמכים שהצליחו

### Batches-טיפול בכשלים ב7.4

1. של 100 מסמכים batch הגישו
2. מצליחים; 5 נכשלים (חרגו מגבול ההקשר) 95
3. custom\_id זהו כשלים לפי
4. (chunks-למשל פצלו מסמכים ארוכים ל) שנו אסטרטגיה
5. הגישו מחדש רק את 5 המסמכים שנכשלו

### SLA תכנון7.5

יכול לקחת עד 24 שעות API Batch-אם צריך תוצאה תוך 30 שעות ו

- חלון הגשה: 30 - 24 = 6 שעות
- deadline-חייבים להיות מוגשים לא יאוחר מ-24 שעות לפני ה Batches
- להגשות תכופות, פצלו לחלונות של 4 שעות



# פרק 8: אסטרטגיות פירוק משימות

## 8.1 Fixed Pipelines (Prompt Chaining)

כל שלב מוגדר מראש:

```
Document -> Metadata extraction -> Data extraction -> Validation -> Enrichment -> Final output
```

מתי להשתמש:

- מבנה המשימה צפוי (ביקורות תמיד עוקבות אחר אותה תבנית)
- כל השלבים ידועים מראש
- צריך יציבות ושחזוריות

## 8.2 Dynamic Adaptive Decomposition

תת-משימות נוצרות על בסיס תוצאות ביניים:

- "Add tests for a legacy codebase"
- > First: map the structure (Glob, Grep)
- > Found: 3 modules with no tests, 2 with partial coverage
- > Prioritize: start with the payments module (high risk)
- > During work: discovered a dependency on an external API
- > Adapt: add a mock for the external API before writing tests

מתי להשתמש:

- משימות חקרניות פתוחות
- כשהיקף מלא לא ידוע מראש
- כשכל שלב תלוי בתוצאות הקודם

## 8.3 Multi-pass Code Review

עם +10 קבצים PRs:

```
Pass 1 (per-file): Analyze auth.ts -> list local issues
Pass 1 (per-file): Analyze database.ts -> list local issues
Pass 1 (per-file): Analyze routes.ts -> list local issues
...
Pass 2 (integration): Analyze relationships between files
-> Cross-file issues: inconsistent types, circular dependencies
```

יחיד מעל 14 קבצים גרוע pass למה:

- דילול תשומת לב: ניתוח עמוק לחלק מהקבצים, שטחי לאחרים
- הערות לא-עקביות: דפוס מסומן בקובץ אחד ומאושר באחר

- באגים מפוספסים: שגיאות ברורות נדלגות עקב עומס קוגניטיבי

# Human-in-the-Loop-פרק 9: הסלמה ו

## מתי להסלים לאדם 9.1

טריגרים להסלמה (כללים ברורים)

| מצב                               | פעולה                                            |
|-----------------------------------|--------------------------------------------------|
| "הלקוח מבקש מפורשות "תביא לי מנהל | הסלימו מיד; אל תנסו לפתור                        |
| המדיניות לא מכסה את הבקשה         | הסלימו (למשל התאמת מחיר למתחרה כשהמדיניות שותקת) |
| הסוכן לא מצליח להתקדם             | הסלימו אחרי מספר ניסיונות סביר                   |
| פעולה פיננסית מעל סף              | (לא בפרומפט, hook רצוי דרך) הסלימו               |
| מספר תוצאות בחיפוש לקוח           | בקשו מזהים נוספים; אל תנחשו                      |

מה לא טריגר אמין

| שיטה לא-אמינה               | למה נכשלת                                       |
|-----------------------------|-------------------------------------------------|
| sentiment ניתוח             | מצב רוח של לקוח לא בקורלציה עם מורכבות התיק     |
| ביטחון עצמי של המודל (1–10) | המודל יכול לטעות בביטחון; קליברציה גרועה        |
| מסווג אוטומטי               | שאינ training data עלול לדרוש; over-engineering |

## דפוסי הסלמה 9.2

הסלמה מיידיית

```
Customer: "I want to speak to a manager"
Agent: [immediately calls escalate_to_human]
NOT: "I can help with your issue, let me..."
```

הסלמה אחרי ניסיון פתרון

```
Customer: "My refrigerator broke two days after purchase"
Agent: [checks the order, offers a warranty replacement]
If the customer is not satisfied -> escalate
```

הסלמה ניואנסית (acknowledge → resolve → escalate on reiteration):

Customer: "This is outrageous, I'm very unhappy with the quality!"  
Agent: [acknowledges frustration] "I understand your frustration."  
[offers resolution] "I can offer a replacement or a refund."  
Customer: "No, I want to talk to someone!"  
Agent: [customer insists again -> immediate escalation]

עיקרון מפתח: הכירו ברגש קודם, אז הציעו פתרון קונקרטי, והסלימו רק אם הלקוח חוזר על הבקשה לבן אדם. אל תסלימו על ביטוי ראשון של חוסר שביעות רצון (זה לא זהה לבקשת מנהל).

**הסלמה על פער במדיניות:**

Customer: "Competitor X has this item 30% cheaper—give me a discount"  
Policy: covers price adjustments only on your own site  
Agent: [escalates – policy does not cover competitor price matching]

## 9.3 מובנים Handoff פרוטוקולי

בהסלמה, הסוכן צריך להעביר סיכום מובנה לאדם:

```
{
  "customer_id": "CUST-12345",
  "customer_name": "Ivan Petrov",
  "issue_summary": "Refund request for a damaged item",
  "order_id": "ORD-67890",
  "root_cause": "Item arrived damaged; photos attached",
  "actions_taken": [
    "Verified customer via get_customer",
    "Confirmed order via lookup_order",
    "Offered a standard replacement – customer insists on a refund"
  ],
  "refund_amount": "$89.99",
  "recommended_action": "Approve a full refund",
  "escalation_reason": "Customer requested to speak with a manager"
}
```

האופרטור האנושי לא רואה את כל תמלול השיחה — הוא רואה רק את הסיכום הזה. לכן הוא חייב להיות שלם ומכיל בתוך עצמו.

## 9.4 Confidence Calibration ופיקוח אנושי

למערכות חילוף נתונים:

1. **ציוני ביטחון ברמת שדה:** המודל מוציא ציון ביטחון לכל שדה מחולץ.
2. **קליברציה:** השתמשו בסטים מתויגים לכיוון ספים
3. **ניתוב:**
  - ביטחון גבוה + דיוק יציב -> עיבוד אוטומטי
  - ביטחון נמוך או מקורות עמומים -> סקירה אנושית

Stratified random sampling:

- שוטף של מדגם audit גם לחילוצים בביטחון גבוה, בצעו
- דיוק מצרפי של 97% יכול להסתיר 40% שגיאות לסוג מסמך מסוים
- נתחו דיוק לפי סוג מסמך ולפי שדה, לא רק כללי

פרק 10: טיפול בשגיאות במערכות רב-סוכניות

10.1 קטגוריות שגיאה

| קטגוריה    | דוגמאות                         | Retryable     | פעולת סוכן                   |
|------------|---------------------------------|---------------|------------------------------|
| Transient  | כשל רשת, 503, Timeout           | כן            | Retry עם exponential backoff |
| Validation | פורמט קלט לא תקין, שדה חובה חסר | לא (תקנו קלט) | שנו בקשה ונסו שוב            |
| Business   | הפרת מדיניות, חציית סף          | לא            | הסבירו למשתמש; הציעו חלופה   |
| Permission | גישה נדחתה                      | לא            | הסלימו                       |

10.2 אנטי-פטרנים בטיפול שגיאות

| אנטי-פטרן                       | בעיה                                      | גישה נכונה                                     |
|---------------------------------|-------------------------------------------|------------------------------------------------|
| "search unavailable" סטטוס גנרי | המתאם לא יכול להחליט איך להתאושש          | תוצאות, query, החזירו סוג שגיאה חלקיות, חלופות |
| השתקה שקטה (תוצאה ריקה = הצלחה) | המתאם חושב שלא היו תוצאות, אבל זה היה כשל | "הבדילו "אין תוצאות" מ"כשל בחיפוש"             |
| ביטול כל הזרימה על כשל אחד      | אתם מאבדים את כל התוצאות החלקיות          | המשיכו עם תוצאות חלקיות; סמנו פערים            |
| אינסופיים בתוך תת- Retries סוכן | ובזבוז משאבים latency                     | אז הפצה, (2-1 retries) החלמה מקומית למתאם      |

## שגיאת תת-סוכן מובנית 10.3

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    {"title": "AI Music Generation Report", "url": "...", "relevance": 0.8}
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

זה מספק למתאם את המידע הדרוש כדי להחליט:

- שונה עם query עם Retry?
- להשתמש בתוצאות חלקיות?
- להאציל לתת-סוכן אחר?
- להמשיך בלי הסקציה הזו ולסמן את הפער?

## הערות כיסוי בסינתזה הסופית 10.4

```
## Report: AI Impact on Creative Industries

### Visual Art (FULL COVERAGE)
[research results]

### Music (PARTIAL COVERAGE – search agent timeout)
[partial results]
⚠ Note: coverage for this section is limited due to a timeout in the search agent.

### Literature (FULL COVERAGE)
[research results]
```

# פרק 11: ניהול הקשר במערכות פרודקשן

## חילוץ עובדות לבלוק נפרד 11.1

במקום להסתמך על היסטוריית שיחה (שיורדת באיכות בעת סיכום), חלצו עובדות מפתח לבלוק מובנה:

```
=== CASE FACTS (updated whenever a new fact appears) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Order Date: 2025-01-15
Order Amount: $89.99
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
```

כללו את הבלוק הזה בכל פרומפט, ללא קשר לאיך ההיסטוריה מסוכמת.

## קיצוץ תוצאות כלי 11.2

מחזיר +40 שדות אבל אתם צריכים רק 5 למשימה הנוכחית `lookup_order` אם

```
# PostToolUse hook: keep only relevant fields
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

זה חוסך הקשר ומפחית רעש.

## קלט מודע-מיקום 11.3

lost-in-the-middle מקמו מידע קריטי תוך התחשבות באפקט

```
[KEY FINDINGS – at the top]
Found 3 critical vulnerabilities...

[DETAILED RESULTS – middle]
=== File auth.ts ===
...
=== File database.ts ===
...

[ACTION ITEMS – at the end]
Priority: fix auth.ts vulnerabilities before merge.
```

## Scratchpad קבצי 11.4

scratchpad בחקירות ארוכות, הסוכן יכול לכתוב ממצאים מרכזיים לקובץ

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() is called from 3 places: OrderController, AdminPanel, CronJob
- External PaymentGateway API has a rate limit of 100 req/min
- Migration #47 added refund_reason (NOT NULL) – 2024-12-01
```

מחדש discovery במקום להריץ scratchpad-כשההקשר יורד באיכותו (או בסשן חדש), הסוכן יכול להתייעץ ב

## האצלה לתת-סוכנים כדי להגן על ההקשר 11.5

```
Main agent: "Investigate dependencies of the payments module"
-> Subagent (Explore): reads 15 files, traces imports
-> Returns: "Payments depends on AuthService, OrderModel, and the external
PaymentGateway API"

Main agent: keeps one line in context instead of 15 files
```

**שכבת הקשר נפרדת:** במערכות רב-סוכניות, כל תת-סוכן פועל בתוך תקציב הקשר מוגבל — הוא מקבל רק את גלובלי, ומקצה state המידע הנדרש למשימתו. המתאם פועל כשכבת הקשר נפרדת: הוא מצרף פלטי תת-סוכן, שומר שבו סוכן אחד צורך את החלון במידע לא רלוונטי לאחרים, "context leakage" הקשר. זה מונע

**תקציבי הקשר מוגבלים לתת-סוכנים:**

- שלחו הקשר מינימלי: משימה ספציפית + נתונים נדרשים
- של נתונים גולמיים dumps הנחו את תת-הסוכן להחזיר תוצאות מובנות, לא
- כדי להגביל את ערכת הכלים של תת-הסוכן — פחות כלים פירושו פחות הסחות `allowedTools`-השתמשו ב דעת ועלות הקשר נמוכה יותר

## מובנה (להתאוששות מקריסה) State שמירת 11.6

שלו למיקום ידוע state-כל סוכן מייצא את ה

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

בהמשך manifest המתאם טוען.

## (מקור הציטוט) Provenance פרק 12: שימור

### 12.1 Attribution בעיית אובדן

בעת סיכום תוצאות ממקורות מרובים, הקישור "טענה" → מקור יכול ללכת לאיבוד:

Bad: "The AI music market is estimated at \$3.2B." (No source, no year.)

Good:

```
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

### 12.2 טיפול בנתונים סותרים

כששני מקורות מספקים ערכים שונים



```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Difference in methodology and time period"
}
```

ותנו למתאם להחליט attribution אל תבחרו ערך אחד שרירותית. שמרו את שניהם עם

## כללו תאריכים לפרשנות נכונה 12.3

בלי תאריכים, הבדלים זמניים יכולים להתפרש בטעות כסתירות:

Bad: "Source A says 10%, source B says 15%. Contradiction."

Good: "Source A (2023) says 10%, source B (2024) says 15%. Likely +5% growth over a year."

## רינדור לפי סוג תוכן 12.4

אל תכפו הכל לפורמט אחד:

- נתונים פיננסיים -> טבלאות
- חדשות וניתוח -> פרוזה
- ממצאים טכניים -> רשימות מובנות
- סדרות זמן -> סדר כרונולוגי

# Claude Code פרק 13: כלים מובנים של

## הפניית בחירת כלי 13.1

| משימה | כלי | דוגמה |
|-------|-----|-------|
|-------|-----|-------|

|                          |       |                                                                  |
|--------------------------|-------|------------------------------------------------------------------|
| מציאת קבצים לפי שם/תבנית | Glob  | <code>**/*.test.tsx</code> , <code>src/components/**/*.ts</code> |
| חיפוש בתוך קבצים         | Grep  | שם פונקציה, הודעת שגיאה, import                                  |
| קריאת קובץ במלואו        | Read  | טעינת קובץ לניתוח                                                |
| כתיבת קובץ חדש           | Write | יצירת קובץ מאפס                                                  |
| עריכת קובץ קיים מדויקת   | Edit  | ספציפי דרך התאמת טקסט ייחודית snippet החלפת                      |
| shell הרצת פקודת         | Bash  | git, npm, בדיקות, build, הרצת                                    |

## 13.2 אסטרטגיית חקירה הדרגתית

אל תקראו את כל הקבצים בבת אחת. בנו הבנה הדרגתית

1. Grep: find entry points (function definition, export)
  2. Read: read the found files
  3. Grep: find usages (import, calls)
  4. Read: read consumer files
  5. Repeat until you have a complete picture

## 13.3 Fallback: Read + Write במקום Edit

נכשל בגלל התאמת טקסט לא-ייחודית Edit-כש:

1. Read — טענו את תוכן הקובץ המלא
2. שנו את התוכן תוכניתית
3. Write — כתבו את הגרסה המעודכנת

# הערות לפי דומיין מבחן: II חלק

## דומיין 1: ארכיטקטורת סוכנים ואורקסטרציה (27%)

### 1.1 תכנון לולאות סוכן לביצוע משימות אוטונומיות

ידע מפתח

- בדקו Claude מחזור חיים של לולאת סוכן: שלחו בקשת `stop_reason` ( מול `"tool_use"` `"end_turn"` ), הריצו כלים, החזירו תוצאות לאיטרציה הבאה
- תוצאות כלי מצורפות להיסטוריית השיחה כדי שהמודל יחליט על הפעולה הבאה
- לעומת עצי החלטה מקודדים-קשיחים (בוחר את הכלי הבא Claude) model-driven קבלת החלטות

## מיומנויות מפתח:

- `stop_reason = "tool_use"` - בעקרת זרימה: המשיכו את הלולאה כש
- `"end_turn"` - ועצרו ב
- צירוף תוצאות כלי להקשר בין איטרציות
- לסיום, שימוש בגבולות איטרציה שרירותיים כמנגנון עצירה ראשי `assistant` אנטי-פטרנים להימנע: ניתוח טקסט

## אורקסטרציה של מערכות רב-סוכניות (מתאם-תת-סוכן) 1.2

### ידע מפתח:

- המתאם מחזיק בכל התקשורת בין סוכנים, טיפול שגיאות וניתוב: `hub-and-spoke` ארכיטקטורת
- תת-סוכנים פועלים עם הקשר מבודד — הם לא יורשים אוטומטית את היסטוריית המתאם
- אחריות המתאם: פירוק משימה, האצלה, צבירת תוצאות, בחירה דינמית של תת-סוכנים
- סיכון של פירוק מצומצם מדי על ידי המתאם

### מיומנויות מפתח:

- חלקו כיסוי מחקרי בין תת-סוכנים כדי למזער כפילות
- מימשו לולאות שיפור איטרטיביות (המתאם מעריך סינתזה ומנתב מחדש משימות)
- נתבו את כל התקשורת דרך המתאם לצורך תצפיתיות

## הגדרת קריאות תת-סוכן, העברת הקשר ויצירה 1.3

### ידע מפתח:

- `"Task"` של המתאם חייב לכלול `allowedTools`; יוצר תת-סוכנים `Task` הכלי
- הקשר תת-סוכן חייב להיכלל מפורשות בפרומפט; תת-סוכנים לא יורשים הקשר הורה
- מגבלות כלים, `system prompts`, תיאורים: `AgentDefinition` הגדרת
- לחקירת חלופות `fork_session` ניהול סשנים דרך

### מיומנויות מפתח:

- כללו פלטים מלאים מסוכנים קודמים בפרומפט תת-הסוכן
- השתמשו בפרומטים מובנים להפרדת נתונים ממטא-דאטה בהעברת הקשר
- בתור מתאם יחיד `Task` צרו תת-סוכנים מקבילים דרך מספר קריאות
- כתבו פרומפטי מתאם במונחי מטרות וקריטריוני איכות במקום הוראות שלב-אחר-שלב

## handoff-מימוש זרימות עבודה רב-שלביות עם אכיפה ו 1.4

### ידע מפתח:

- לבין הנחיית פרומפט לסדר זרימת עבודה (`hooks`, `preconditions`) ההבדל בין אכיפה תוכנית
- כשצריך הבטחות דטרמיניסטיות (למשל אימות זהות לפני פעולות פיננסיות), פרומפטים לבדם לא מספיקים
- מובנים בהסלמה (זיהוי לקוח, סיבה, פעולה מומלצת) `handoff` פרוטוקולי

## מיומנויות מפתח:

- למשל חסום) תוכנותיים שחוסמים קריאות במורד הזרימה עד שצעדים קודמים הושלמו Preconditions (מאומת ID מחזיר `get_customer` -עד ש `process_refund`)
- פרקו בקשות לקוח רב-היבטיות לפריטים נפרדים
- הפיקו סיכומים מובנים בהסלמה לאדם

## ליירוט קריאות כלי ונורמליזציה Agent SDK ב Hooks 1.5

### ידע מפתח:

- ליירוט תוצאות כלי לפני שהמודל צורך אותן (`PostToolUse` למשל) Hook דפוסי
- למשל חסום החזרים מעל סף) `compliance` שמייטרים קריאות יוצאות לאכיפת כללי Hooks
- הסתברותי `compliance` מספקים הבטחות דטרמיניסטיות לעומת הוראות פרומפט שמספקות Hooks

### מיומנויות מפתח:

- (קודי סטטוס מספריים, ISO 8601, Unix timestamps) לנורמליזציית פורמטי נתונים `hooks` `PostToolUse`
- ליירוט שחוסמים פעולות שמפרות מדיניות עם ניתוב להסלמה Hooks
- מובטח `compliance` על פני פרומפטים כשכללי עסק דורשים `hooks` בחרו

## אסטרטגיות פירוק משימות לזרימות מורכבות 1.6

### ידע מפתח:

- לעומת פירוק אדפטיבי דינמי מבוסס תוצאות ביניים) `Fixed pipelines` (prompt chaining)
- Prompt chaining: (אינטגרציה `pass` נתחו כל קובץ בנפרד, ואז הריצו) צעדים סדרתיים
- תוכניות חקירה אדפטיביות שיוצרות תת-משימות לפי מה שהתגלה

### מיומנויות מפתח:

- לסקירות צפויות רב-היבטיות; השתמשו בפירוק דינמי לחקירות פתוחות `prompt chaining`-השתמשו ב
- אינטגרציה נפרד לבין-קבצים `pass` גדולים לניתוח לפי קובץ ועוד `code reviews` פצלו
- פרקו משימות פתוחות: מפו מבנה קודם, אז בנו תוכנית מתועדפת

## Session State, Resume I-Fork 1.7

### ידע מפתח:

- להמשך ששנים בעלי שם `--resume <session-name>`
- ליצירת ענפי חקירה עצמאיים מהקשר משותף `fork_session`
- חשיבות יידוע הסוכן על שינויי קבצים בעת המשך ששנים
- סשן חדש עם סיכום מובנה יכול להיות אמין יותר מהמשך עם תוצאות מיושנות

## מיומנויות מפתח:

- להמשך סשני חקירה בעלי שם `--resume` -השתמשו ב
- להשוואת גישות במקביל `fork_session` -השתמשו ב
- לעומת התחלת סשן חדש (תוצאות מיושנות) (הקשר עדיין רלוונטי) `resume` בחרו בין

## MCP (18%) דומיין 2: עיצוב כלים ואינטגרציית

### עיצוב ממשקי כלי עם תיאורים ברורים 2.1

#### ידע מפתח:

- משתמש בו לבחירת כלים; תיאורים מינימליים מובילים לבחירה לא-LLM-תיאורי כלי הם המנגנון העיקרי ש אמינה
- חשיבות הכללת פורמטי קלט, שאילתות לדוגמה, מקרי קצה וגבולות תחולה
- תיאורים דו-משמעיים או חופפים גורמים לניתוב שגוי
- יכול ליצור אסוציאציות לא-רצויות לכלים `system prompt` ניסוח

#### מיומנויות מפתח:

- כתבו תיאורים שמבדילים בבירור כל כלי מחלופות דומות
- (`analyze_content` -> `extract_web_results`) למשל) שנו שמות כלים כדי לבטל חפיפה פונקציונלית
- פצלו כלים כלליים למתמחים עם חוזי קלט/פלט ברורים

### MCP מימוש תגובות שגיאה מובנות לכלי 2.2

#### ידע מפתח:

- MCP בתגובות כלי `isError` הדגל
- הפרות **business שגיאות**, (קלט שגוי) **validation שגיאות**, (timeouts) **transient** ההבדל בין **שגיאות** ושגיאות גישה/הרשאה, (מדיניות מונעות החלטות התאוששות נכונות ("Operation failed") שגיאות גנריות
- `retryable` ל-`non-retryable` ההבדל בין שגיאות

#### מיומנויות מפתח:

- והודעה, `isRetryable`, (`transient/validation/permission`) `errorCategory` החזירו מטא-דאטה מובנית כמו קריאה לאדם
- עם הסברים ברורים למשתמש `business-rule` להפרות `retryable: false` -השתמשו ב
- הפיצו רק שגיאות שהם לא יכולים לפתור; `transient` בצעו החלמה מקומית בתוך תת-סוכנים לכשלים
- מתוצאות ריקות תקפות (אין תוצאות) (`retry` החלטת) הבדילו כשלי גישה

## 2.3 הקצאת כלים בין סוכנים והגדרת tool\_choice

### ידע מפתח:

- יותר מדי כלים לסוכן (למשל 18 במקום 4-5) **מפחית** אמינות בחירת כלים
- סוכנים עם כלים מחוץ להתמחותם נוטים להשתמש בהם לרעה
- בין-תפקידים utilities גישת כלים מצומצמת: רק כלים רלוונטיים לתפקיד ועוד ערכה מוגבלת של
- `tool_choice`: `"auto"`, `"any"`, ובחירה כפויה, `{"type": "tool", "name": "..."} (`

### מיומנויות מפתח:

- הגבילו את ערכת הכלים של כל תת-סוכן למה שרלוונטי לתפקידו
- `fetch_url` -> `load_document` (למשל) החליפו כלים כלליים בחלופות מוגבלות
- להבטחת קריאת כלי במקום תשובה טקסטואלית `tool_choice: "any"` -השתמשו ב
- כפו כלי ספציפי כדי להבטיח סדר ביצוע

## 2.4 וזרימות סוכן Claude Code ב-MCP שילוב שרתי

### ידע מפתח:

- לניסיונות ( `~/.claude.json` ) לצוותים לעומת משתמש ( `.mcp.json` ) פרויקט MCP: היקף שרת
- לניהול סודות ( `_${GITHUB_TOKEN}` ) (למשל) `.mcp.json` -החלפת משתני סביבה ב
- המחברים מתגלים בחיבור וזמינים בו-זמנית MCP כלים מכל שרתי
- להפחתת קריאות כלי חקרניות (DB סיכומי משימות, סכמות) "כ"קטלוגי תוכן MCP resources

### מיומנויות מפתח:

- `env-var` של הפרויקט עם טוקנים מבוססי `.mcp.json` -משותפים ב MCP הגדירו שרתי
- `~/.claude.json` -שמרו שרתים אישיים/ניסיוניים ב
- קהילתיים על פני שרתים מותאמים אישית לאינטגרציות סטנדרטיות MCP העדיפו שרתי

## 2.5 בחירה ויישום של כלים מובנים (Read, Write, Edit, Bash, Grep, Glob)

### ידע מפתח:

- `Grep` (imports, שמות פונקציות, הודעות שגיאה) חיפוש בתוך תוכן קבצים
- `Glob`: מציאת קבצים לפי תבניות שם/סיומת
- `Read/Write`: פעולות מלאות על קבצים; `Edit`: ייחודיות טקסט
- `Read + Write` ל-fallback, נכשל עקב התאמות לא-ייחודיות Edit אם

### מיומנויות מפתח:

- לגילוי קבצים לפי תבניות Glob לחיפוש תוכן וב Grep -השתמשו ב
- flows לעקיבת Read אז, Grep entry points: בנו הבנה הדרגתית
- wrapper עקבו אחר שימוש בפונקציות דרך מודולי

# וזרימות עבודה (20%) Claude Code דומיין 3: הגדרות

## עם היררכיה, היקף וארגון מודולרי CLAUDE.md הגדרת 3.1

### ידע מפתח:

- או `.claude/CLAUDE.md` (פרויקט), `~/claude/CLAUDE.md` (משתמש CLAUDE.md: היררכיית CLAUDE.md (בתת-תיקיות CLAUDE.md) ורמת תיקיה, (בשורש CLAUDE.md
- VCS הגדרות ברמת משתמש חלות רק על משתמש אחד ולא משותפות דרך
- למודולריות (`./standards/coding-style.md` (למשל) להתייחסות לקבצים חיצוניים `@path` תחביר CLAUDE.md
- מונוליטי CLAUDE.md לקבצי כללים ממוקדי-נושא במקום `.claude/rules/` תיקיית

### מיומנויות מפתח:

- אבחנו בעיות היררכיה (חבר צוות חדש מפספס הוראות כי הן ברמת משתמש במקום פרויקט)
- בכל חבילה standards להכללה סלקטיבית של (`./standards/testing.md` (למשל) `@path` -השתמשו ב
- גדול לקבצי CLAUDE.md פצלו (`testing.md`, `api-conventions.md`, `deployment.md`) מרובים `.claude/rules/` גדול לקבצי CLAUDE.md פצלו

## 3.2 Custom Slash Commands ויצירה והגדרה של Skills

### ידע מפתח:

- -לעומת פקודות משתמש ב (VCS משותפות דרך) `.claude/commands/` -פקודות פרויקט ב `~/claude/commands/`
- Skills `.claude/skills/` של frontmatter עם `SKILL.md`: `context: fork`, `allowed-tools`, `argument-hint`
- בהקשר תת-סוכן מבודד כדי שלא יזהם את הסשן הראשי skill-מריץ את ה `context: fork`
- בשמות שונים `~/claude/skills/` -אישיים יכולים לחיות ב skills ואריאנטי

### מיומנויות מפתח:

- כדי שכל הצוות יקבל אותן `.claude/commands/` -של פרויקט ב slash commands שמרו
- verbose עם פלט skills לבידוד `context: fork` -השתמשו ב
- יכול להשתמש בהם skill להגבלת אילו כלים `allowed-tools` -השתמשו ב
- כדי לדרבן מפתחים לפרמטרים נדרשים `argument-hint` -השתמשו ב

## 3.3 לטעינת מוסכמות מותנית Path Specific שימוש בכללי

### ידע מפתח:

- glob להפעלת כללים על בסיס תבניות `paths` עם frontmatter YAML יכולים לכלול `.claude/rules/` קבצי

- נטענים רק בעריכת קבצים תואמים, חוסכים הקשר וטוקנים path-כללים מסומני
- ברמת תיקייה כשמוסכמות חלות על הרבה CLAUDE.md יכולים להיות עדיפים על glob-מבוססי path כללי (tests למשל) תיקיות

### מיומנויות מפתח:

- לטעינה רק בעבודה על קבצים תואמים `paths: ["terraform/**/*"]` עם `.claude/rules/` צרו קבצי
- להחלת מוסכמות לפי סוג קובץ ללא תלות במיקום (glob `(**/*.test.tsx)` השתמשו בתבניות
- codebase-ברמת תיקייה כשמוסכמות מתפרסות על ה CLAUDE.md על path-העדיפו כללים מסומני

## מתי להשתמש במצב תכנון מול ביצוע ישיר 3.4

### ידע מפתח:

- **מצב תכנון:** למשימות מורכבות עם שינויים גדולים, מספר גישות סבירות, והחלטות ארכיטקטוניות
- (בודד validation למשל הוספת) **ביצוע ישיר:** לשינויים פשוטים ומובנים היטב
- לפני ביצוע שינויים codebase-מצב תכנון מאפשר חקירה בטוחה של ה
- מילולי discovery מבודד פלט Explore subagent

### מיומנויות מפתח:

- (מיגרציות שנוגעות ל-45+ קבצים, microservices) השתמשו במצב תכנון למשימות עם השלכות ארכיטקטוניות
- ברור וקובץ יחיד stack trace השתמשו בביצוע ישיר לתיקונים עם
- כדי למנוע אזילת חלון הקשר במשימות רב-שלביות Explore subagent-השתמשו ב
- שלבו גישות: תכנון לגילוי, אז ביצוע ליישום

## שיפור איטרטיבי לשיפור פרוגרסיבי 3.5

### ידע מפתח:

- דוגמאות קונקרטיות של קלט/פלט הן הדרך האפקטיבית ביותר לתקשר ציפיות
- על בסיס כשלים iterate **איטרציה מונחית-בדיקות:** כתבו בדיקות קודם, אז
- שואל שאלות כדי להעלות שיקולי עיצוב לא-מובנים Claude: "interview"דפוס ה
- מתי לספק את כל הבעיות בהודעה אחת (תלויות הדדית) לעומת ברצף (עצמאיות)

### מיומנויות מפתח:

- ספקו 2–3 דוגמאות קונקרטיות של קלט/פלט להבהרת דרישות טרנספורמציה
- בנו ערכות בדיקה עם התנהגות צפויה, מקרי קצה ודרישות ביצועים לפני מימוש
- (מצבי כשל, cache invalidation) להעלאת היבטי עיצוב interview השתמשו בדפוס
- קונקרטיים עם קלטים לדוגמה ופלטים צפויים למקרי קצה test cases ספקו



## 3.6 CI/CD בפייפליין Claude Code שילוב

### ידע מפתח:

- למצב לא-אינטראקטיבי בפייפליין אוטומטי ( `--print` ) (או `-p` ) הדגל
- CI-לפלט מובנה ב `--json-schema` ו `--output-format json`
- CI-שמופעל מ Claude Code ל- (review סטנדרטי בדיקות, קריטריוני) מספק הקשר פרויקט CLAUDE.md
- עצמאי instance- **בידוד הקשר ששן**: אותו ששן שייצר קוד הוא פחות אפקטיבי בסקירתו מ

### מיומנויות מפתח:

- כדי להימנע מתליה על קלט אינטראקטיבי `-p` עם CI ב- Claude Code הריצו
- (PR על inline למשל הערות) לתוצאות מובנות `--json-schema` + `--output-format json` -השתמשו ב
- קודמות בעת הרצה מחדש אחרי קומיטים חדשים (דווחו רק על בעיות חדשות/לא-מתוקנות) review כללו תוצאות
- לשיפור איכות יצירת בדיקות CLAUDE.md-זמינים ב fixtures-תעדו סטנדרטי בדיקות ו
- כללו קבצי בדיקה קיימים בהקשר בעת יצירת בדיקות חדשות כדי להימנע מכפילות ולשמור על עקביות סגנון

## דומיין 4: הנדסת פרומפט ופלט מובנה (20%)

### עיצוב פרומפטים עם קריטריונים מפורשים לשיפור דיוק 4.1

#### ידע מפתח:

- קריטריונים מפורשים אפקטיביים יותר מהוראות מעורפלות (למשל "סמן הערות רק כשהן סותרות את הקוד" מול "בדוק דיוק הערות")
- הנחיה גנרית כמו "תהיה שמרני יותר" עובדת פחות טוב מקריטריונים קטגוריאליים קונקרטיים
- גבוהים בקטגוריות מסוימות פוגעים באמון false-positive על אמון מפתחים: שיעורי false positives ההשפעה של בקטגוריות מדויקות

#### מיומנויות מפתח:

- מה לדווח (באגים, אבטחה) לעומת מה להתעלם (סגנון מינורי): review הגדירו קריטריוני
- גבוהים false-positive בטלו זמנית קטגוריות עם שיעורי
- הגדירו קריטריוני חומרה מפורשים עם דוגמאות קוד לכל רמה

### לשיפור עקביות פלט Few-shot Prompting שימוש ב 4.2

#### ידע מפתח:

- הן השיטה האפקטיבית ביותר להפקת פלט עקבי בפורמט וניתן לפעולה few-shot דוגמאות
- יכול להדגים טיפול במקרים דו-משמעיים (בחירת כלי, פערים בכיסוי בדיקות) Few-shot
- עוזר למודל להכליל לדפוסים חדשים במקום רק לחזור על ברירות מחדל Few-shot
- יכול להפחית הזיות במשימות חילוץ Few-shot

## מיומנויות מפתח:

- ספקו 2–4 דוגמאות ממוקדות לתרחישים דו-משמעיים עם הצדקה
- שמדימות את פורמט הפלט (מיקום, בעיה, חומרה, תיקון מוצע) few-shot כללו דוגמאות
- ספקו דוגמאות שמבדילות דפוסי קוד מקובלים מבעיות אמיתיות
- ספקו דוגמאות לחילוף נכון ממסמכים עם מבנים שונים

## 4.3 I-JSON Schemas ו tool\_use אכיפת פלט מובנה עם

### ידע מפתח:

- הוא הדרך האמינה ביותר להבטיח פלט תואם-סכמה ולבטל שגיאות תחביר JSON Schemas עם tool\_use JSON
- הוא חייב לקרוא לכלי; בחירה כפויה "any" המודל יכול להחזיר טקסט; עם tool\_choice: "auto" עם בוחרת כלי ספציפי
- מחמירים מבטלים שגיאות תחביר אך לא מונעים שגיאות סמנטיות (סכומים לא מסתכמים; JSON Schemas ערכים בשדות שגויים)
- ועוד שדה פירוט להרחבה "other" עם enums; optional לעומת required עיצוב סכמה: שדות

### מיומנויות מפתח:

- ופרסרו נתונים מתוצאות JSON Schemas הגדירו כלי חילוף עם tool\_use
- להבטחת פלט מובנה כשקיימות מספר סכמות tool\_choice: "any" -השתמשו ב
- כפו קריאה לכלי ספציפי: tool\_choice: {"type": "tool", "name": "extract\_metadata"}
- כשהמקור עשוי לא להכיל מידע, כדי להימנע מבדיית ערכים nullable/הפכו שדות לאופציונליים
- ועוד שדות פירוט לקטגוריזציה הניתנת להרחבה "other" ו- "unclear" כמו enum השתמשו בערכי

## 4.4 ולולאות פידבק לאיכות חילוף Validation, Retries מימוש

### ידע מפתח:

- Retry-with-error-feedback: לכוון תיקונים retry-קונקרטיות בפרומפט ה validation כללו שגיאות
- Retries לא אפקטיביים כשהמידע פשוט נעדר מהמקור
- ( detected\_pattern ) עיצוב לולאת פידבק: עקבו אחר הדפוס שהפעיל ממצא
- ( tool\_use ) מטופלות על ידי שגיאות סמנטיות (סכומים לא מסתדרים) לעומת שגיאות תחביר

### מיומנויות מפתח:

- ספציפיות validation עם המסמך המקורי, חילוף שגוי ושגיאות follow-up פרומפטי
- לא יהיה אפקטיבי (המידע הנדרש רק במסמך חיצוני) retry זהו מתי
- false positives בממצאים לניתוח detected\_pattern כללו שדות
- לזיהוי אי-התאמות stated\_total וגם calculated\_total על ידי חילוף גם self-correction עצבו

## Batch עיצוב אסטרטגיות יעילות לעיבוד 4.5

### ידע מפתח:

- Message Batches API: latency על SLA חיסכון, חלון עיבוד עד 24 שעות, אין הבטחות 50%
- בדיקות לפני) ולא מתאים למשימות חוסמות (audits, דוחות לילה) מתאים למשימות לא-חוסמות batch עיבוד merge)
- בתוך בקשה יחידה multi-turn tool calling-לא תומך ב Batch API
- batches בתוך request/response מקשרים custom\_id שדות

### מיומנויות מפתח:

- לעומסי לילה/שבועיים Batch API-סינכרוני לבדיקות חוסמות; השתמשו ב API-השתמשו ב
- למשל חלונות של 4 שעות להבטחת 30 שעות עם עיבוד 24) SLA על בסיס צרכי batch הגשת cadence תכננו (שעות)
- (custom\_id מזהים לפי) טפלו בכשלים על ידי הגשה מחדש רק של מסמכים שנכשלו
- על פרומפטים תוך שימוש במדגם לפני הרצת עיבוד בקנה מידה גדול iterate

## review רב-instance רב עיצוב ארכיטקטורות 4.6

### ידע מפתח:

- המודל שומר את הקשר השיקול שלו ופחות סביר שיאתגר את ההחלטות של עצמו: self-review מגבלות
- ביקורת עצמאיים (בלי הקשר היצירה) טובים יותר במציאת בעיות עדינות Instances
- Multi-pass review: אינטגרציה בין-קבצים כדי להימנע מדילול תשומת לב pass ניתוח מקומי לכל קובץ ועוד

### מיומנויות מפתח:

- שני עצמאי לסקירת שינויים בלי הקשר היצירה Claude instance-השתמשו ב
- אינטגרציה לניתוח זרימת נתונים בין-קבצים passes לפי קובץ ועוד passes-פצלו ביקורות רב-קבציות ל
- אימות עם ביטחון עצמי לניתוב ביקורות באופן מכויל passes-השתמשו ב

## דומיין 5: ניהול הקשר ואמינות (15%)

### ניהול הקשר שיחה לשימור מידע קריטי 5.1

### ידע מפתח:

- סיכוני סיכום פרוגרסיבי: ערכים מספריים, אחוזים ותאריכים מתעבים לסיכומים מעורפלים
- מודלים מעבדים אמינות התחלה וסוף של קלטים ארוכים, אבל עלולים לפספס lost-in-the-middle אפקט ממצאים מהאמצע
- פלטי כלי יכולים להצטבר בהקשר באופן לא-פרופורציוני לרלוונטיות (+40 שדות כש-5 נחוצים)
- עוקבות API חשיבות שליחת היסטוריית השיחה המלאה בבקשות

## מיומנויות מפתח:

- קבוע מחוץ להיסטוריה המסוכמת "case facts" חלצו עובדות טרנזקציוניות לבלוק
- לשדות רלוונטיים verbose קצצו פלטי כלי
- מקמו ממצאי מפתח בתחילת נתונים מצורפים עם כותרות סקציה מפורשות
- דרשו שתת-סוכנים יכללו מטא-דאטה (תאריכים, מקורות) בפלטים מובנים

## עיצוב דפוס הסלמה אפקטיביים ופתרון דו-משמעיות 5.2

### ידע מפתח:

- טריגרי הסלמה מתאימים: בקשה מפורשת לאדם, פערים/חריגים במדיניות, חוסר יכולת להתקדם
- הסלמה מיידית (בקשה מפורשת) לעומת ניסיון פתרון (בתחום הסוכן)
- וביטחון עצמי של המודל הם פרוקסי לא-אמינים למורכבות תיק sentiment ניתוח
- מספר התאמות לקוח דורש בקשת מזהים נוספים, לא ניחוש היוריסטי

### מיומנויות מפתח:

- few-shot ב-system prompt קריטריוני הסלמה מפורשים עם דוגמאות
- בצעו בקשות מפורשות לאדם מיד ללא חקירה נוספת
- הסלימו כשהמדיניות דו-משמעית או שותקת לבקשה ספציפית
- בקשו מזהים נוספים כשתוצאות כלי מכילות מספר התאמות

## מימוש אסטרטגיות הפצת שגיאות במערכות רב-סוכניות 5.3

### ידע מפתח:

- מאפשר התאוששות חכמה יותר של מתאם (תוצאות חלקיות, חלופות, query, סוג כשל) הקשר שגיאה מובנה
- מתוצאות ריקות תקפות (אין התאמות) (retry דורשים החלטת timeouts) הבדילו כשלי גישה
- מסתירים הקשר חשוב מהמתאם ("search unavailable") סטטוסי שגיאה גנריים
- השתקה שקטה או ביטול כל הזרימה על כשל אחד הם אנטי-פטרנים

### מיומנויות מפתח:

- החזירו הקשר שגיאה מובנה: סוג כשל, מה נוסה, תוצאות חלקיות, חלופות אפשריות
- הבדילו כשלי גישה מתוצאות ריקות תקפות
- הפיצו רק שגיאות לא-ניתנות-להתאוששות עם תוצאות; transient בצעו החלמה מקומית בתת-סוכנים לכשלים חלקיות
- סמנו כיסוי בסניתזה: מה מבוסס היטב לעומת היכן נשאר פער

## גדולים Codebases ניהול הקשר יעיל בחקירת 5.4

### ידע מפתח:

- ירידה באיכות הקשר בסשנים ארוכים: המודל מתחיל להפיק תשובות לא-יציבות ומתייחס ל"דפוסים אופייניים" במקום למחלקות ספציפיות

- משמרים ממצאי מפתח מעבר לגבולות הקשר Scratchpad קבצי
- מילולי discovery האצלה לתת-סוכנים מבודדת פלט
- מובנה מאפשרת התאוששות מקריסה state שמירת

### מיומנויות מפתח:

- צרו תת-סוכנים לשאלות ספציפיות תוך שמירה על תיאום ברמה גבוהה בסוכן הראשי
- לאחסון ממצאי מפתח והפניה אליהם בהמשך scratchpad השתמשו בקבצי
- סכמו ממצאי מפתח לפני יצירת תת-סוכני שלב הבא
- להפחתת שימוש בהקשר בחקירות ארוכות `/compact` -השתמשו ב

## עיצוב זרימות עם פיקוח אנושי וקליברציית ביטחון 5.5

---

### ידע מפתח:

- מדדים מצרפיים (למשל 97% דיוק כללי) יכולים להסוות ביצועים גרועים בסוגי מסמכים או שדות ספציפיים
- מודד שיעורי שגיאה בחילוצים בביטחון גבוה Stratified random sampling
- קליברציית ביטחון ברמת שדה תוך שימוש בסטים מתויגים
- אמתו דיוק לפי סוג מסמך ומגזר שדה לפני אוטומציה

### מיומנויות מפתח:

- לזיהוי דפוסי שגיאה חדשים stratified random sampling מימשו
- נתחו דיוק לפי סוג מסמך ושדה לאימות ביצועים יציבים
- תוך שימוש בנתונים מתויגים review הוציאו ציוני ביטחון ברמת שדה וכיילו ספי
- נתבו חילוצים בביטחון נמוך או ממקורות עמומים לסקירה אנושית

## טיפול באי-ודאות בסינתזה רב-מקורית Provenance שימור 5.6

---

### ידע מפתח:

- "אובד בסיכום ללא שימור מיפוי "טענה → מקור Attribution
- מיפויים מובנים חייבים להישמר בעת צבירה
- במקום בחירה שרירותית של ערך אחד attribution טפלו בסטטיסטיקות סותרות על ידי הערה על קונפליקטים עם
- כללו תאריכי פרסום/איסוף כדי להימנע מקריאה שגויה של הבדלים זמניים כסתירות

### מיומנויות מפתח:

- (שם מסמך, ציטוטים, URL) "דרשו שתת-סוכנים יוציאו מיפוי "טענה → מקור
  - בנו דוחות שמפרידים ממצאים יציבים מאלה הנמצאים במחלוקת
  - שמרו ערכים סותרים עם הערות והעבירו אותם למתאם להתאמה
  - כללו תאריכי פרסום לפרשנות זמנית נכונה
  - הנדרו תוכן לפי סוג: נתונים פיננסיים כטבלאות, חדשות כפרוזה, ממצאים טכניים כרשימות מובנות
-

# דוגמאות שאלות מבחן עם הסברים

## 1 שאלה (תרחיש: Customer Support Agent)

רק עם שם lookup\_order וקורא ל get\_customer מצב: הנתונים מראים שב-12% מהמקרים הסוכן מדלג על הלקוח, מה שמוביל להחזרים שגויים.

איזה שינוי הכי אפקטיבי

- A) מ-ID עד שמתקבל lookup\_order I- process\_refund תוכנתי שחוסם precondition הוסיפו A) [נכון] get\_customer
- B) system prompt שפרו את ה B)
- C) הוסיפו דוגמאות few-shot C)
- D) מימשו מסווג ניתוב D)

כשלוגיקה עסקית קריטית דורשת רצף כלים מסוים, תוכנה מספקת הבטחות דטרמיניסטיות שגישות: A למה מטפל בזמינות, לא בסדר כלים D. לא יכולות B, C) מבוססות-פרומפט

## 2 שאלה (תרחיש: Customer Support Agent)

לשאלות הקשורות בהזמנה. תיאורי הכלים lookup\_order -במקום ל get\_customer -מצב: הסוכן קורא ל מינימליים ודומים.

מה הצעד הראשון

- A) few-shot דוגמאות A)
- B) הרחיבו את תיאור כל כלי עם פורמטי קלט, דוגמאות וגבולות [נכון] B)
- C) הוסיפו שכבת ניתוב C)
- D) מזגו את הכלים D)

תיאורי כלים הם מנגנון הבחירה העיקרי של המודל. זה התיקון בעלות הנמוכה ביותר עם ההשפעה הגבוהה: B למה דורש יותר מאמץ מהמוצדק D. over-engineering זה C. מוסיף טוקנים בלי לטפל בשורש הבעיה A. ביותר

## 3 שאלה (תרחיש: Customer Support Agent)

מצב: הסוכן פותר רק 55% מהבעיות במטרת 80%. הוא מסלים מקרים פשוטים ומנסה לטפל באופן אוטונומי בחריגי מדיניות מורכבים.

איך משפרים את הקליברציה

- A) few-shot הוסיפו קריטריוני הסלמה מפורשים עם דוגמאות A) [נכון]
- B) ביטחון עצמי (1–10) עם הסלמה אוטומטית B)
- C) מסווג נפרד שאומן על נתונים היסטוריים C)
- D) sentiment ניתוח D)

זה C. לא אמין (המודל יכול לטעות בביטחון) B. מטפל ישירות בשורש הבעיה — גבולות החלטה לא-ברורים A: למה פותר בעיה אחרת (מצב רוח != מורכבות) D. over-engineering.

## 4 שאלה (תרחיש: Code Generation with Claude Code)

של הריפו clone סטנדרטי שזמינה לכל הצוות בעת code review ל-`/review` מצב: אתם צריכים פקודה מותאמת.

איפה ליצור את קובץ הפקודה?

- A) `.claude/commands/` הפרויקט [נכון] ב-repo
- B) `~/ .claude/commands/`
- C) `CLAUDE.md` בשורש
- D) `.claude/config.json`

B. זמינות אוטומטית לכולם version-controlled הן `.claude/commands/` -פקודות פרויקט הנשמרות ב: A: למה לא קיים D. להוראות, לא להגדרות פקודה C. לפקודות אישיות.

## 5 שאלה (תרחיש: Code Generation with Claude Code)

(עשרות קבצים, החלטות גבולות שירות) microservices-מצב: צריך לבצע ארגון מחדש של מונולית ל

באיזה גישה להשתמש?

- A) הבינו תלויות, עצבו גישה [נכון], codebase-מצב תכנון: חקרו את ה
- B) ביצוע ישיר הדרגתי
- C) ביצוע ישיר עם הוראות מפורטות מראש
- D) ביצוע ישיר ומעבר לתכנון כשנהיה קשה

מסכן בעבודה חוזרת B. מצב תכנון מתוכנן לשינויים גדולים, מספר גישות אפשריות והחלטות ארכיטקטוניות A: למה ריאקטיבי D. מניח שאתם כבר יודעים את המבנה C. יקרה.

## 6 שאלה (תרחיש: Code Generation with Claude Code)

בדיקות ממוקמות יחד עם הקוד. (React, API, database) יש מוסכמות שונות באזורים שונים codebase-מצב: ל אתם רוצים שמוסכמות יוחלו אוטומטית.

באיזה גישה להשתמש?

- A) קבצי `.claude/rules/` עם YAML frontmatter glob ותבניות [נכון]
- B) בשורש `CLAUDE.md` שמו הכל ב
- C) Skills ב- `.claude/skills/`
- D) בכל תיקייה `CLAUDE.md`

מאפשר החלת מוסכמות אוטומטית על ( `**/*.test.tsx` למשל) glob עם תבניות `.claude/rules/` A: למה לא D. ידני/לפי דרישה C. מסתמך על הסקת המודל B. codebase-בסיס נתיבי קבצים — אידיאלי לבדיקות פזורות ב

עובד טוב כשקבצים רלוונטיים בהרבה תיקיות.

## 7 (תרחיש: Multi-agent Research System) שאלה

על תעשיות יצירה", אבל הדוחות מכסים רק אמנות חזותית. המתאם פירק את AI מצב: המערכת חוקרת "השפעת "בצילום AI", "בעיצוב גרפי AI", "באמנות דיגיטלית AI": הנושא ל

מה הסיבה

- A) סוכן הסינתזה לא מזהה פערים
- B) המתאם פירק את המשימה בצורה מדי [נכון]
- C) סוכן חיפוש האינטרנט לא מחפש מספיק יסודית
- D) סוכן ניתוח המסמכים מסנן מקורות לא-חזותיים

הלוגים מראים שהמתאם פירק את "תעשיות יצירה" רק לתת-נושאים חזותיים, מפספס לגמרי מוזיקה, B: למה ספרות וקולנוע. תת-הסוכנים ביצעו נכון — הבעיה היא במה שהוקצה להם.

## שאלות נוספות

הקובץ המקורי כולל למעלה מ-70 שאלות. המבנה והעקרונות זהים — מקרי שימוש מתרחשי המבחן, ארבע לעיל. II-I אפשרויות, והסבר למה התשובה הנכונה היא הנכונה. כל אלה מבוססים על העקרונות שכוסו בחלקים [practical\\_test\\_en.html](#) לתרגול מלא, ראו את

מומלץ להתאמן בשאלות באנגלית כיוון שכך תקבלו את ניסוח השאלות הזהה למבחן האמיתי.

## תרגילים מעשיים

### תרגיל 1: סוכן רב-כלים עם לוגיקת הסלמה

מטרה: לתכנן לולאת סוכן עם אינטגרציית כלים, טיפול שגיאות מובנה והסלמה

שלבים:

- עם תיאורים מפורטים (כללו שני כלים דומים לבחינת בחירת כלי) MCP הגדירו 3-4 כלי.
- מימשו לולאת סוכן הבודקת ( "tool\_use" / "end\_turn" ) stop\_reason
- תיאור, errorCategory, isRetryable: הוסיפו תגובות שגיאה מובנות
- ליירוט שחוסם פעולות מעל סף ומנתב להסלמה hook מימשו
- בדקו עם בקשות רב-היבטיות

(הקשר ואמינות) 5, (MCP-כלים I) דומיינים: 1 (ארכיטקטורת סוכן), 2



## לפיתוח צוותי Claude Code תרגיל 2: הגדרת

MCP ספציפיים ושרתי path פקודות מותאמות אישית, כללי, CLAUDE.md **מטרה:** להגדיר

**שלבים:**

1. universal ברמת פרויקט עם סטנדרטי CLAUDE.md צרו
  2. ( paths: ["src/api/\*\*/\*"] , paths: ["\*\*/\*.test.\*"] ) לאזורי קוד שונים YAML frontmatter עם .claude/rules/ צרו קבצי
  3. context: fork I- allowed-tools עם .claude/skills/ פרויקט תחת skill צרו
  4. override + עם משתני סביבה .mcp.json -ב MCP הגדירו שרת ~/.claude.json אישי
  5. בדקו מצב תכנון לעומת ביצוע ישיר במשימות בעלות מורכבות שונה
- 3 (דומיינים: 2 (MCP-כלים ו), 2 (Claude Code הגדרות), 3

## לחילוץ נתונים מובנים Pipeline: תרגיל 3

batch עיבוד, validation/retry, לולאות tool\_use, JSON schemas, **מטרה:**

**שלבים:**

1. (JSON schema הגדירו כלי חילוץ עם nullable שדות, "other" עם enums, required/optional שדות)
  2. ספציפית validation עם המסמך, חילוץ שגוי ושגיאת retry, בשגיאה: validation: בנו לולאת
  3. למסמכים עם מבנים שונים few-shot הוסיפו דוגמאות
  4. מסמכים, טיפול בכשלים דרך Message Batches API: 100 דרך batch השתמשו בעיבוד custom\_id
  5. ניתוב לאדם: ציוני ביטחון ברמת שדה, ניתוח לפי סוג מסמך
- 4 (הנדסת פרומפט), 5 (הקשר ואמינות) **דומיינים:**

## מחקרי רב-סוכני pipeline של debugging-תרגיל 4: עיצוב ו

**מטרה:** אורקסטרציית תת-סוכנים, העברת הקשר, הפצת שגיאות, סינתזה עם מעקב מקורות

**שלבים:**

1. (הקשר מועבר מפורשות בפרומפטים, "Task" כולל allowedTools ) מתאם עם +2 תת-סוכנים
  2. בתגובה אחת Task הריצו תת-סוכנים במקביל דרך מספר קריאות
  3. מקור, תאריך פרסום URL, דרשו פלט תת-סוכן מובנה: טענה, ציטוט
  4. של תת-סוכן: החזירו הקשר שגיאה מובנה למתאם והמשיכו עם תוצאות חלקיות timeout דמו
  5. הפרידו ממצאים מאומתים ממוכרזים; attribution; בדקו עם נתונים סותרים: שמרו את שני הערכים עם
- 2 (הקשר ואמינות), 5 (MCP-כלים ו), 1 (ארכיטקטורת סוכן), **דומיינים:**

# נספח: טכנולוגיות ומושגים

| טכנולוגיה                    | היבטי מפתח                                                                                                                                  |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Claude Agent SDK             | AgentDefinition, agent loops, stop_reason , hooks (PostToolUse), יצירת תת-סוכנים דרך Task, allowedTools                                     |
| Model Context Protocol (MCP) | משתני סביבה , mcp.json , תיאורי כלים , isError , MCP, tools, resources שרתי                                                                 |
| Claude Code                  | CLAUDE.md, glob עם תבניות , .claude/rules/ , .claude/commands/ , .claude/skills/ עם SKILL.md, מצב תכנון, /compact , --resume , fork_session |
| Claude Code CLI              | -p / --print למצב לא-אינטראקטיבי , --output-format json , --json-schema                                                                     |
| Claude API                   | tool_use עם JSON schemas, tool_choice ("auto"/"any"/"forced"), stop_reason , max_tokens , system prompts                                    |
| Message Batches API          | multi-turn tool calling אין , custom_id , חיסכון, חלון של עד 24 שעות 50%                                                                    |
| JSON Schema                  | Required לעומת optional, nullable שדות, enum סוגי, "other" + מצב strict פירוט,                                                              |
| Pydantic                     | validation/retry אימות סכמה, שגיאות סמנטיות, לולאות                                                                                         |
| כלים מובנים                  | Read, Write, Edit, Bash, Grep, Glob — מטרה וקריטריוני בחירה                                                                                 |
| Few-shot prompting           | דוגמאות ממוקדות למצבים דו-משמעיים, הכללה לדפוסים חדשים                                                                                      |
| Prompt chaining              | ממוקדים passes-פירוק סדרתי ל                                                                                                                |
| חלון הקשר                    | scratchpad קבצי, "lost in the middle", תקציבי טוקנים, סיכום פרוגרסיבי                                                                       |
| ניהול ששנים                  | Resume, fork_session , ששנים בעלי שם, בידוד הקשר,                                                                                           |
| קליברציית ביטחון             | stratified sampling, ניקוד ברמת שדה, קליברציה על סטים מתווגים                                                                               |

## נושאים מחוץ להיקף

- הנושאים הסמוכים הבאים לא יהיו במבחן
- או אימון מודלים מותאמים Claude של מודלי Fine-tuning
  - חיוב או ניהול חשבון Claude API, אימות
  - tool/schema מעבר למה שדרוש להגדרת) ספציפיים frameworks מימוש מפורט בשפות תכנות או
  - (תשתית, רשת) MCP פריסה או אירוח של שרתי container orchestration
  - תהליך אימון או משקלי מודל Claude, ארכיטקטורה פנימית של
  - או מתודולוגיות אימון בטיחות RLHF, Constitutional AI
  - vector database או embedding פרטי מימוש של מודלי

- Computer use (אוטומציית דפדפן, אינטראקציה דסקטופ)
- Vision) יכולות ניתוח תמונה
- Streaming API או server-sent events
- Rate limiting, או חישובי עלות, מפורטים API מכסות,
- OAuth, סבב, API keys, פרטי פרטוקול אימות,
- (AWS, GCP, Azure) הגדרות ספציפיות לספק ענן
- בנצ'מרקים של ביצועים או מדדי השוואת מודלים
- (מעבר לידיעה שזה קיים) prompt caching פרטי מימוש של
- tokenization אלגוריתמי ספירת טוקנים או פרטי

## המלצות הכנה

1. מימשו לולאת סוכן מלאה עם קריאה לכלים, טיפול שגיאות וניהול ששנים. — **Claude Agent SDK בנו סוכן עם**. תתאמנו בתת-סוכנים והעברת הקשר מפורשת
2. ספציפיים path כללי, CLAUDE.md **לפרויקט אמיתי** — השתמשו בהיררכיית **Claude Code** הגדירו MCP ושילוב שרת, `allowed-tools` I- `context: fork` עם `skills`, `claude/rules/` -ב
3. retry, כתבו תיאורים שמבדילים כלים דומים, החזירו שגיאות מובנות עם קטגוריות ודגלי — **MCP תכננו ובדקו כלי**. ובדקו מול בקשות משתמש דו-משמעיות
4. שדות, validation/retry, לולאות, JSON schemas עם `tool_use` -**לחילוף נתונים** — השתמשו ב **pipeline בנו** optional/nullable, Message Batches API דרך batch ועיבוד
5. מפורשים, review לתרחישים דו-משמעיים, קריטריוני few-shot **תתאמנו בהנדסת פרומפט** — הוסיפו דוגמאות גדולים code reviews ל-code multi-pass וארכיטקטורות
6. discovery והאצילו, scratchpad **למדו דפוסי ניהול הקשר** — חלצו עובדות מפלטים מילוליים, השתמשו בקבצי. לתת-סוכנים כדי לטפל בגבולות הקשר
7. מתי להסלים (פערי מדיניות, בקשה מפורשת של משתמש, חוסר יכולת — **human-in-the-loop-הבינו הסלמה ו**. להתקדם) וזרימות ניתוב מבוסס-ביטחון
8. **קחו מבחן תרגול** לפני האמיתי. הוא משתמש באותם תרחישים ופורמט.

מותר להישראל, `guide_en.MD` **על התרגום:** המדריך הזה הוא תרגום עברי של המדריך האנגלי  נשמרו באנגלית לפי המוסכמה הישראלית. ('וכו, `tool_use`, MCP, CLAUDE.md) טכני. מונחים טכניים API נשמרו באנגלית כיוון שהם תחביר enum דוגמאות קוד, סכמות וערכי

[GitHub Issues](#) תרומות לשיפור התרגום מתקבלות בברכה דרך